CH01

Chapter 1: Setting Up Your Go Environment

SECTIONS COVERED

Installing the Go Tools

Troubleshooting Your Go Installation

Go Tooling

Your First Go Program

Making a Go Module

go build

go fmt

go vet

Choose Your Tools

Visual Studio Code

GoLand

The Go Playground

Makefiles

The Go Compatibility Promise

Staying Up-to-Date

Exercises

Wrapping Up

GOAL

Set up a Go development environment, create a module, and build, format, and vet a Go program.

KEY IDEAS

1. Install Go tools for your platform from golang.org/dl.

```
go version
```

2. A Go project is a module defined by a go.mod file.

```
go mod init hello_world
```

3. go fmt enforces one standard code format.

```
go fmt ./...
```

4. go vet scans valid code for likely bugs.

```
go vet ./...
```

5. go build compiles a program to a single native binary.

```
go build
```

FULL NOTES

Installing the Go Tools

- RULE: Download and install the Go development tools for your platform from the Go website.
- Mac uses .pkg, Windows uses .msi, Linux/BSD use gzipped TAR files.
- Add /usr/local/go/bin to your $PATH so the go command is accessible.
- GOTCHA: Forgetting to add Go to your PATH, so the go command is not found.

Troubleshooting Your Go Installation

- RULE: If go version errors, go is likely not in your executable path or another program named go is in the path.
- Use which go on macOS/Unix to see if the go command resolves.
- On Linux/BSD, check you installed the 64-bit tools for your chip architecture.
- GOTCHA: Installing 64-bit Go tools on a 32-bit system or wrong architecture.

Go Tooling

- RULE: All Go development tools are accessed via the go command.
- Tools include go build, go fmt, go mod, go test, go vet, and more.
- Chapters 10, 11, and 15 cover the tools in detail.
- GOTCHA: none in text

Your First Go Program

- RULE: Every Go program starts from the main function in the main package.
- Code is organized into packages; imports reference whole packages only.
- All Go programs start from func main().
- GOTCHA: none in text

Making a Go Module

- RULE: Create a directory, then run go mod init to mark it as a Go module.
- go mod init creates a go.mod file in the module root.
- go.mod declares the module name, minimum Go version, and dependencies.
- Don't edit go.mod directly; use go get and go mod tidy.
- GOTCHA: Editing go.mod directly instead of using go get/go mod tidy.

go build

- RULE: go build compiles code into an executable named after the module.
- Use the -o flag to name or relocate the binary.
- Go programs compile to a single native binary with no extra software needed to run.
- GOTCHA: none in text

go fmt

- RULE: go fmt fixes whitespace to match Go's standard format.
- Go uses tabs to indent; opening brace must be on the same line as the declaration.
- Run go fmt before compiling and before committing source changes.
- GOTCHA: Putting the opening brace on the wrong line (go fmt won't fix it).

go vet

- RULE: go vet detects bugs in code that is syntactically valid but likely incorrect.
- It checks formatting templates have a value for every placeholder.
- Run go vet alongside go fmt to keep code quality high.
- GOTCHA: Assuming go vet catches every possible bug; it does not.

Choose Your Tools

- RULE: Go IDEs and editors provide formatting on save, completion, type checking, and debugging.
- Visual Studio Code and GoLand are two popular Go development environments.
- VS Code needs the Go extension; GoLand works without a plug-in.
- GOTCHA: none in text

Visual Studio Code

- RULE: Install the Go extension from the VS Code marketplace for Go support.
- The Go extension installs Delve (debugger) and gopls (language server).
- You must install the Go compiler yourself.
- GOTCHA: none in text

GoLand

- RULE: GoLand is JetBrains' Go-specific IDE with refactoring, completion, debugging, and coverage.
- GoLand includes JavaScript/HTML/CSS and SQL tools; no plug-in required.
- GoLand is commercial; a free license program and 30-day trial exist.
- GOTCHA: none in text

The Go Playground

- RULE: The Go Playground lets you try and share small programs without installing Go.
- Simulate multiple files with -- filename.go -- separators.
- Network access is limited to localhost; long-running or heavy processes are stopped.
- The clock is fixed to November 10, 2009, 23:00:00 UTC.
- GOTCHA: Putting sensitive information (passwords, private keys) into the playground.

Makefiles

- RULE: Makefiles specify repeatable build steps with targets and dependencies.
- Indent target steps with a tab; .DEFAULT_GOAL sets the default target.
- .PHONY prevents make confusion when a file shares a target name.
- Makefiles are picky about tabs and not supported out-of-the-box on Windows.
- GOTCHA: Using spaces instead of a tab to indent Makefile target steps.

The Go Compatibility Promise

- RULE: No backward-breaking changes to the language or standard library for Go 1.x versions, except bug/security fixes.
- The guarantee does not apply to go command flags and functionality.
- GOTCHA: none in text

Staying Up-to-Date

- RULE: Programs compiled with different Go versions can run simultaneously on one machine.
- Mac/Windows users update via brew, chocolatey, or the latest installer.
- Linux/BSD users move the old version aside, unpack the new one, then delete the old.
- GOTCHA: none in text

Exercises

- RULE: Each chapter ends with exercises; answers are in the chapter repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You can now install and configure Go and use tools to build and ensure code quality.
- Next chapter covers built-in types and variable declaration.

- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
go version	Check installed Go version
go mod init name	Create a module and go.mod
go build	Compile to a native binary
go build -o name	Name or relocate the binary
go fmt ./...	Format code to Go standard
go vet ./...	Scan valid code for likely bugs
go mod tidy	Manage go.mod dependencies

SELF-TEST

1. Which command creates a go.mod file for a new module?
2. Which command fixes code whitespace to match Go's standard format?
3. Which command scans syntactically valid code for likely bugs?
4. What character must indent steps in a Makefile target?
5. Where should you never put sensitive information when using The Go Playground?

ANSWERS

1. go mod init name
2. go fmt ./...
3. go vet ./...
4. A tab
5. The Go Playground (saved on Google's servers)

CH02

Chapter 2: Predeclared Types and Declarations

SECTIONS COVERED

The Predeclared Types

The Zero Value

Literals

Booleans

Numeric Types

A Taste of Strings and Runes

Explicit Type Conversion

Literals Are Untyped

var Versus :=

Using const

Typed and Untyped Constants

Unused Variables

Unused Constants

Naming Variables and Constants

Exercises

Wrapping Up

GOAL

Use Go's built-in types and declare variables, constants, and literals idiomatically.

KEY IDEAS

1. Every declared-but-unassigned variable gets its type's zero value.

```
var flag bool
```

2. Go has no automatic type promotion; convert types explicitly.

```
float64(x) + y
```

3. Literals are untyped and work with any compatible type.

```
var x float64 = 10
```

4. Use := inside functions; var at package level.

```
x := 10
```

5. Constants name compile-time literals; they cannot hold runtime values.

```
const x = 10
```

FULL NOTES

The Predeclared Types

- RULE: Go's built-in types are called predeclared types: booleans, integers, floats, strings, runes, bytes.
- Write code so your intentions are clear.
- GOTCHA: none in text

The Zero Value

- RULE: Any declared-but-unassigned variable gets its type's default zero value.
- Zero value makes code clearer and removes bugs common in C and C++.
- GOTCHA: none in text

Literals

- RULE: A literal is an explicitly specified number, character, or string.
- Integer literals: base 10 default; 0b binary, 0o octal, 0x hex; avoid leading-0 octal.
- Underscores may group digits (1_234) but not at start/end or adjacent.
- Rune literals use single quotes ('a'); double quotes and single quotes are not interchangeable.

- Use interpreted (double-quote) strings most of the time; use raw (backtick) strings for backslashes/quotes/newlines.
- GOTCHA: A leading 0 with no letter is an octal literal and is very confusing.

Booleans

- RULE: The bool type has exactly two values: true and false; its zero value is false.
- GOTCHA: none in text

Numeric Types

- RULE: Go has signed and unsigned integer types from one to eight bytes (int8…int64, uint8…uint64).
- byte is an alias for uint8; prefer byte. rune is an alias for int32.
- int and uint are platform-sized; int cannot be mixed with int32/int64 without conversion.
- Choose int unless a specific size/sign or generics is needed.
- Integer division truncates toward zero; dividing an integer by 0 panics.
- Float types: float32 and float64; use float64 unless you must match an existing format.
- Floats cannot store every value exactly; never use them for money.
- Don't compare floats with ==; use an epsilon (max allowed variance).
- Complex types: complex64 and complex128 via the complex built-in; extract parts with real and imag.
- GOTCHA: Using floats for money or comparing floats with ==.

A Taste of Strings and Runes

- RULE: The zero value for a string is the empty string; strings are immutable.
- Strings compare with ==/!=/< and concatenate with +.
- rune is an alias for int32; use rune when you mean a character.
- GOTCHA: none in text

Explicit Type Conversion

- RULE: Go has no automatic type promotion; use a type conversion when variable types differ.
- This applies even to different-sized integers and floats.
- No other type can be converted to bool, implicitly or explicitly; use comparison operators.
- GOTCHA: Treating a nonzero number or nonempty string as a boolean (truthiness).

Literals Are Untyped

- RULE: Literals have no fixed type and work with any compatible variable type.
- You cannot assign a string literal to a numeric variable, a number literal to a string, or a float literal to an int.
- A literal that overflows its target variable (e.g., 1000 into a byte) is a compile-time error.
- GOTCHA: Assigning an integer literal that overflows the target type, such as 1000 to a byte.

var Versus :=

- RULE: var is the verbose declaration; := is short declaration with type inference, only inside functions.
- := cannot be used at package level; use var there.
- To signal a zero value is intended, use var x int rather than x := 0.
- := can assign to existing variables as long as at least one new variable is on the left.
- GOTCHA: Using := and accidentally creating a new variable when you meant to reuse an existing one (shadowing).

Using const

- RULE: const names compile-time literals; it can only hold values the compiler knows at compile time.
- Allowed: numeric literals, true/false, strings, runes, complex/real/imag/len/cap results, and expressions of these.

- const cannot hold a value calculated at runtime.
- There is no way to declare a variable immutable; arrays, slices, maps, and structs are not immutable.
- GOTCHA: Declaring const z = x + y from runtime variables (won't compile).

Typed and Untyped Constants

- RULE: Constants can be typed (const typedX int = 10) or untyped (const x = 10).
- Untyped constants act like literals and are more flexible; typed constants enforce a type.
- A typed constant assigns directly only to a variable of that type.
- GOTCHA: Assigning a typed constant to a different type (compile-time error).

Unused Variables

- RULE: Every declared local variable must be read; otherwise it's a compile-time error.
- The check is not exhaustive: a variable read once won't complain even if other writes are unread.
- Package-level unread variables are allowed; another reason to avoid them.
- GOTCHA: Declaring a local variable and never reading it.

Unused Constants

- RULE: The compiler allows unread constants because they are compile-time and side-effect free.
- An unused constant is simply not included in the compiled binary.
- GOTCHA: none in text

Naming Variables and Constants

- RULE: Identifiers start with a letter or underscore and may contain letters, numbers, underscores; any Unicode letter/digit is allowed.
- Idiomatic Go uses camel case (indexCounter), not snake case.
- Don't use type information in variable names; use short names in small scopes, descriptive names in the package block.
- An underscore alone (_) is a special identifier.
- Constants are not all-uppercase with underscores in Go.
- GOTCHA: Using look-alike Unicode code points that look the same but are different variables.

Exercises

- RULE: Chapter exercises let you try the concepts; solutions are in the Chapter 2 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You now understand predeclared types, variables, assignments, and operators.
- The next chapter covers composite types: arrays, slices, maps, and structs.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
var x int	Declare x with zero value
x := 10	Short declare with inference (in functions only)
const x = 10	Name a compile-time literal
float64(x)	Explicit type conversion
byte	Alias for uint8
rune	Alias for int32

'a'	Rune literal

SELF-TEST

1. What is the zero value of a bool?
2. What keyword is used inside functions for short variable declaration?
3. What keyword must be used to declare variables at package level?
4. Which numeric type should you use unless you need a specific size or format?
5. Can a float literal be assigned to an int variable?

ANSWERS

1. false
2. :=
3. var
4. float64
5. No

CH03

Chapter 3: Composite Types

SECTIONS COVERED

Arrays-Too Rigid to Use Directly

Slices

len

append

Capacity

make

Emptying a Slice

Declaring Your Slice

Slicing Slices

copy

Converting Arrays to Slices

Converting Slices to Arrays

Strings and Runes and Bytes

Maps

Reading and Writing a Map

The comma ok Idiom

Deleting from Maps

Emptying a Map

Comparing Maps

Using Maps as Sets

Structs

Anonymous Structs

Comparing and Converting Structs

Exercises

Wrapping Up

GOAL

Use Go's composite types: arrays, slices, maps, strings, and structs.

KEY IDEAS

1. Arrays are rigid; size is part of the type, so prefer slices.

```
var x []int
```

2. append returns a new slice; always assign the result back.

```
x = append(x, 10)
```

3. Slices share memory; slicing does not copy data.

```
y := x[:2]
```

4. Maps return the zero value for missing keys; use comma ok to check presence.

```
v, ok := m["k"]
```

5. Struct fields are accessed with dot notation.

```
bob.name = "Bob"
```

FULL NOTES

Arrays-Too Rigid to Use Directly

- RULE: Arrays have a fixed size that is part of the type; [3]int and [4]int are different types.
- Declare with var x [3]int or a literal var x = [3]int{10, 20, 30}.
- You cannot use a variable for the array size or convert arrays of different sizes.
- Compare arrays with ==/!= (same length and equal values).
- Use arrays only when you know the exact length ahead of time (e.g., checksums).
- GOTCHA: Reading/writing past the end of an array panics at runtime.

Slices

- RULE: A slice's length is not part of its type, so slices can grow and work at any size.
- Use [] for a slice, [...] for an array; var x = []int{10, 20, 30}.
- A nil slice's zero value is nil; a nil slice contains nothing.
- Slices are not comparable; only compare a slice to nil with ==.
- Use slices.Equal / slices.EqualFunc (Go 1.21+) to compare slices; avoid reflect.DeepEqual.
- GOTCHA: Comparing two slices with == is a compile-time error.

len

- RULE: len returns the number of elements in an array, slice, string, map, or channel.
- Passing a nil slice to len returns 0.
- len is a built-in; you cannot write your own function accepting all those types.
- GOTCHA: none in text

append

- RULE: append grows a slice and returns a new slice that you must assign back.
- append takes a slice and a value of the same type; you can append multiple values.
- Appending one slice to another uses the ... operator: append(x, y...).
- Forgetting to assign append's result is a compile-time error.
- GOTCHA: Not assigning the result of append back to the slice.

Capacity

- RULE: Every slice has a length (assigned elements) and a capacity (reserved memory).
- cap returns the current capacity; len returns the current length.
- When length reaches capacity, append allocates a larger backing array and copies data.
- As of Go 1.18, capacity doubles below 256, then grows by (cap+768)/4.
- GOTCHA: none in text

make

- RULE: make creates a slice with type, length, and optional capacity: make([]int, 5, 10).
- make([]int, 5) gives length 5 and capacity 5, all zero values.
- make([]int, 0, 10) gives a non-nil slice, length 0, capacity 10; append to it.
- Never specify a capacity less than the length (compile-time or panic).
- GOTCHA: Populating make-initialized elements with append (adds after the zero values).

Emptying a Slice

- RULE: clear(s) sets all slice elements to their zero value; length stays unchanged.
- Go 1.21 added the clear built-in.
- GOTCHA: none in text

Declaring Your Slice

- RULE: Choose a declaration style that minimizes slice growth.
- Use a nil slice (var data []int) if the slice might not grow.
- Avoid []int{} (zero-length, zero-capacity) because it compares unequal to nil.
- Use make with nonzero length as a buffer; use make with zero length and capacity for append.
- append always increases length; don't append after setting length with make unless intended.
- GOTCHA: Using []int{} instead of a nil slice (confusingly different from nil).

Slicing Slices

- RULE: A slice expression x[low:high] shares memory; it does not copy data.
- Omitting low defaults to 0; omitting high defaults to the slice length.
- Changes to an element affect all slices sharing that element.
- A subslice's capacity is the original capacity minus the starting offset.
- GOTCHA: Modifying a slice after slicing it (changes are reflected in both).

copy

- RULE: copy(dst, src) copies elements from source to destination and returns the count copied.
- It copies as many as fit, limited by the smaller length; capacity doesn't matter.

- You don't need to assign copy's return value if you don't need the count.
- copy works between slices and between arrays (via slicing the array).
- Use copy to make an independent slice.
- GOTCHA: none in text

Converting Arrays to Slices

- RULE: Convert an array to a slice with a slice expression; xArray[:] converts the whole array.
- Slicing an array shares memory with the array (same as slicing a slice).
- GOTCHA: none in text

Converting Slices to Arrays

- RULE: Convert a slice to an array with a type conversion: [4]int(xSlice).
- The data is copied to new memory; changes to one don't affect the other.
- The array size must be a compile-time constant; [...] is not allowed in the conversion.
- The array can't be bigger than the slice's length; bigger causes a runtime panic.
- Converting a slice to a pointer to an array (*[4]int)(xSlice) shares storage.
- GOTCHA: Converting a slice to an array bigger than the slice's length (panics).

Strings and Runes and Bytes

- RULE: A string is a sequence of bytes, usually UTF-8-encoded code points.
- Indexing/slicing a string counts bytes, not runes; len returns byte length.
- A UTF-8 code point can be 1 to 4 bytes; slicing mid-character yields invalid data.
- Only slice/index strings when you know they contain single-byte characters.
- Convert string to []byte or []rune; convert a rune or byte to string with string().
- Converting an int to string gives a Unicode character (e.g., 65 -> "A"), not the digits.
- go vet blocks string conversion from integer types other than rune or byte (Go 1.15+).
- Use the strings and unicode/utf8 packages instead of slice/index on strings.
- GOTCHA: Slicing a string at a byte boundary that splits a multi-byte character.

Maps

- RULE: A map is written map[keyType]valueType; the zero value is nil.
- Reading a nil map returns the value type's zero value; writing to a nil map panics.
- Create an empty (non-nil) map with map[string]int{} or make(map[string]int, 10).
- Maps grow automatically; len returns the number of key-value pairs.
- Maps are not comparable; only compare a map to nil with ==.
- Map keys must be comparable; slices and maps cannot be keys.
- Use slices for ordered/sequential data; maps for looking up by a name or key.
- GOTCHA: Writing to a nil map (panics).

Reading and Writing a Map

- RULE: Assign with m["key"] = value and read with m["key"]; you cannot use := on a map key.
- Reading a missing key returns the zero value for the value type.
- The ++ operator works on map values even when no value exists yet (zero value + 1).
- GOTCHA: none in text

The comma ok Idiom

- RULE: v, ok := m["key"] returns the value and a bool indicating whether the key exists.
- ok true means the key is present; ok false means it is absent.
- Use it to distinguish a stored zero value from a missing key.
- Also used when reading from channels and in type assertions.

- GOTCHA: none in text

Deleting from Maps

- RULE: delete(m, "key") removes the key-value pair; it does not return a value.
- Deleting a missing key or deleting from a nil map does nothing.
- GOTCHA: none in text

Emptying a Map

- RULE: clear(m) sets a map's length to zero (unlike a cleared slice).
- GOTCHA: none in text

Comparing Maps

- RULE: Maps cannot be compared with ==/!=; only compare a map to nil.
- Use maps.Equal and maps.EqualFunc (Go 1.21+) to check if two maps are equal.
- GOTCHA: Comparing two maps with ==.

Using Maps as Sets

- RULE: Simulate a set with a map where keys are the set values and values are bool.
- Duplicate keys collapse; len tells you the number of unique values.
- Reading a missing key returns false (the bool zero value).
- Use struct{} as the value to save memory (zero bytes), but it makes code clumsier.
- GOTCHA: none in text

Structs

- RULE: A struct groups related fields; define with type name struct { field type }.
- Field name comes before field type; no commas between fields in the declaration.
- A zero-value struct has every field set to its field's zero value.
- Access fields with dot notation: bob.name = "Bob".
- Struct literals: positional (all fields, in order) or named (any order, optional fields).
- You cannot mix positional and named styles in one literal.
- GOTCHA: Mixing positional and named struct literal styles.

Anonymous Structs

- RULE: An anonymous struct has no type name; declare inline with var person struct { ... }.
- Assign and read fields the same way as a named struct.
- Handy for translating external data (JSON) and for table-driven tests.
- GOTCHA: none in text

Comparing and Converting Structs

- RULE: A struct is comparable only if all its fields are comparable (no slices, maps, functions, channels).
- Structs of different types cannot be compared with == even if fields look the same.
- You can convert one struct type to another if fields match in names, order, and types.
- Anonymous structs can be compared/assigned to identical named structs without conversion.
- GOTCHA: Comparing structs of different types with ==.

Exercises

- RULE: Chapter exercises test composite types; solutions are in the Chapter 3 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You now know slices, maps, strings, and structs; next up is control structures.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
var x []int	Declare a nil slice
x = append(x, v)	Grow a slice (assign result back)
cap(x)	Return slice capacity
make([]int, 5, 10)	Make slice with length and capacity
clear(s)	Zero all elements, keep length
y := x[:2:2]	Full slice expression; cap is 2
copy(dst, src)	Copy elements; return count
v, ok := m["k"]	Comma ok; check key presence
delete(m, "k")	Remove a key-value pair
bob.name	Access a struct field

SELF-TEST

1. Why should you prefer slices over arrays in Go?
2. What must you do with the value returned by append?
3. What does a map return when you read a key that was never set?
4. What does the comma ok idiom's second value indicate?
5. Can two slices be compared with ==?

ANSWERS

1. Array size is part of its type; slices can be any size
2. Assign it back to the slice
3. The zero value for the value type
4. Whether the key is present in the map
5. No

CH04

Chapter 4: Blocks, Shadows, and Control Structures

SECTIONS COVERED

Blocks

Shadowing Variables

if

The Complete for Statement

The Condition-Only for Statement
The Infinite for Statement
break and continue
The for-range Statement
Labeling Your for Statements
Choosing the Right for Statement
switch
Blank Switches
Choosing Between if and switch
goto-Yes, goto
Exercises
Wrapping Up

GOAL

Use blocks, if/for/switch control structures, and avoid variable shadowing.

KEY IDEAS

1. A variable declared in an inner block shadows the outer-block variable of the same name.

```
x := 5
```

2. for-range loops over runes (not bytes) in a string.

```
for _, r := range s
```

3. Go switch cases don't fall through by default.

```
switch size := len(word); size
```

4. Use goto only rarely to jump to a label within the same block.

```
goto done
```

5. An if can declare a variable scoped to the if/else blocks.

```
if n := rand.Intn(10); n == 0
```

FULL NOTES

Blocks

- RULE: Each place a declaration occurs is a block; variables declared outside functions are in the package block.
- You can access an identifier from any outer block within an inner block.
- GOTCHA: none in text

Shadowing Variables

- RULE: Declaring a variable with the same name as one in a containing block shadows the outer variable.
- While shadowed, you cannot access the outer variable; it isn't gone or reassigned.
- := reuses only variables declared in the current block; an outer variable on the left side is shadowed.
- Shadowing can apply to package imports (e.g., a local fmt variable shadows the fmt package).
- Built-in types and constants live in the universe block and can be shadowed; never redefine them.
- go vet does not report shadowing as a likely error.

- GOTCHA: Accidentally shadowing a variable when using :=.

if

- RULE: Go's if has no parentheses around the condition.
- An if can declare a variable scoped to the condition and both if/else blocks: if n := ...; n == 0.
- Once the if/else ends, such a variable is undefined outside.
- Don't put non-variable-declaring statements before the comparison; use it only to define new scoped variables.
- GOTCHA: Referencing an if-scoped variable after the if/else ends (undefined).

The Complete for Statement

- RULE: The C-style for is for init; condition; post { } with no parentheses.
- The init must use := (var is not legal); the post can be any assignment.
- You can leave out init, condition, or post.
- GOTCHA: none in text

The Condition-Only for Statement

- RULE: Omitting init and post (and semicolons) gives a while-like loop: for i < 100 { }.
- GOTCHA: none in text

The Infinite for Statement

- RULE: for { } loops forever; combine with if and break to simulate do/while.
- The loop body should always contain break or return; bound iteration and fail gracefully.
- GOTCHA: none in text

break and continue

- RULE: break exits the loop immediately; continue skips to the next iteration.
- break/continue apply to the directly containing loop unless a label is used.
- Use continue to keep if bodies short and left-aligned instead of nested if/else chains.
- GOTCHA: none in text

The for-range Statement

- RULE: for-range iterates over arrays, slices, strings, maps, and channels.
- You get two variables: position (index/key) and value; use _ to ignore the position.
- You can omit the value and keep only the key: for k := range m.
- Map iteration order varies randomly (a security feature against Hash DoS).
- Iterating a string with for-range gives runes, not bytes, and skips multibyte offsets.
- The for-range value is a copy; modifying it won't modify the source.
- Pre-Go 1.22 reused one index/value variable per loop; Go 1.22+ creates new ones each iteration (control via go directive).
- GOTCHA: Modifying the for-range value variable expecting it to change the source.

Labeling Your for Statements

- RULE: Labels let break/continue target an outer loop in nested loops.
- Labels are indented to the same level as the surrounding braces.
- GOTCHA: none in text

Choosing the Right for Statement

- RULE: Favor for-range when walking all contents of a built-in compound type.
- Use a standard for loop when not iterating first-to-last (clearer start and end).
- Don't use a standard for loop to skip leading runes in a string (it mishandles multibyte chars); use for-range.

- GOTCHA: none in text

switch

- RULE: switch has no parentheses around the compared value and cases don't fall through by default.
- A variable declared in switch is scoped to all branches; case block variables are visible only in that case.
- Match multiple values with commas: case 1, 2, 3.
- An empty case means nothing happens.
- Use fallthrough sparingly; reconsider your logic instead.
- You can switch on any type comparable with == (not slices, maps, channels, functions, or structs with such fields).
- A break in a switch case exits the case, not an enclosing for loop; use break label to exit the loop.
- GOTCHA: Assuming switch cases fall through like C.

Blank Switches

- RULE: A blank switch (switch { }) lets each case use any boolean comparison.
- Prefer an expression switch when all cases compare the same variable for equality.
- Use switch over if/else chains when you have multiple related cases.
- GOTCHA: none in text

Choosing Between if and switch

- RULE: A switch indicates a relationship between the cases; use if/else for unrelated comparisons.
- GOTCHA: none in text

goto-Yes, goto

- RULE: goto jumps to a labeled line; Go forbids jumps that skip variable declarations or enter inner/parallel blocks.
- Use goto rarely; labeled break/continue handle most nested-loop exits.
- goto can clarify code when you can't restructure logic (e.g., run cleanup no matter why you left a loop).
- GOTCHA: Jumping over a variable declaration or into an inner block (compile error).

Exercises

- RULE: Chapter exercises apply blocks, shadows, and control structures; answers in the Chapter 4 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You can now write simple Go programs within main; next is functions to organize larger programs.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
if n := f(); n == 0 {}	Declare variable scoped to if/else
for i := 0; i < 10; i++ {}	Complete C-style for loop
for i < 100 {}	Condition-only (while-like) loop
for { break }	Infinite loop with exit
for _, v := range s {}	Loop over index and value
for k := range m {}	Loop over map keys only
switch v { case 1: }	Switch; no fall-through by default

switch { case x > 0: }	Blank switch with boolean cases
break outer	Break to a labeled outer loop
goto label	Jump to a label (use rarely)

SELF-TEST

1. What happens when you declare a variable with the same name as one in an outer block?
2. What does a for-range loop iterate over when given a string?
3. Do Go switch cases fall through by default?
4. What variable do you use to ignore a for-range value you don't need?
5. Can break exit an enclosing for loop from inside a switch without a label?

ANSWERS

1. It shadows the outer-block variable
2. Runes
3. No
4. Underscore (_)
5. No; use a labeled break

CH05

Chapter 5: Functions

SECTIONS COVERED

Declaring and Calling Functions

Simulating Named and Optional Parameters

Variadic Input Parameters and Slices

Multiple Return Values

Multiple Return Values Are Multiple Values

Ignoring Returned Values

Named Return Values

Blank Returns-Never Use These!

Functions Are Values

Function Type Declarations

Anonymous Functions

Closures

Passing Functions as Parameters

Returning Functions from Functions

defer

Go Is Call by Value

Exercises

Wrapping Up

GOAL

Write and call Go functions, use multiple returns, closures, defer, and treat functions as values.

KEY IDEAS

1. Go functions can return multiple values; error is conventionally the last one.

```
return num / denom, num % denom, nil
```

2. A closure reads and writes variables from its enclosing function.

```
a = 30
```

3. defer delays a call until the surrounding function exits; multiple defers run LIFO.

```
defer f.Close()
```

4. Functions are values; assign and pass them like any other value.

```
var f func(string) int
```

5. Go is call by value; maps and slices behave like references because they hold pointers.

```
modifyFails(i, s, p)
```

FULL NOTES

Declaring and Calling Functions

- RULE: A function declaration has func, name, input parameters, and return type; parameter name comes before type.
- When parameters are consecutive and same-type, specify the type once: func div(num, denom int) int.
- If a function returns a value, you must supply a return; if it returns nothing, no return needed at the end.
- GOTCHA: none in text

Simulating Named and Optional Parameters

- RULE: Go doesn't have named or optional parameters; simulate with a struct passed to the function.
- A function shouldn't have more than a few parameters; many inputs means it's too complicated.
- GOTCHA: none in text

Variadic Input Parameters and Slices

- RULE: A variadic parameter is last/only, written ...type, and becomes a slice inside the function.
- You can pass any number of values or none; to pass a slice, add ... after it: addTo(3, a...).
- Forgetting ... when passing a slice to a variadic parameter is a compile-time error.
- GOTCHA: Passing a slice to a variadic parameter without the ... suffix.

Multiple Return Values

- RULE: Multiple return types are listed in parentheses; return all values separated by commas (no extra parentheses).
- By convention, the error is the last (or only) value returned; return nil for success.
- GOTCHA: Putting parentheses around multiple returned values (compile-time error).

Multiple Return Values Are Multiple Values

- RULE: You must assign each returned value to a variable; assigning multiple returns to one variable is a compile-time error.
- GOTCHA: Assigning multiple return values to a single variable.

Ignoring Returned Values

- RULE: Assign unused returned values to _; use _ whenever you don't need to read a returned value.
- Go lets you implicitly ignore all return values, but in almost all other cases make it explicit with _.
- GOTCHA: none in text

Named Return Values

- RULE: Name return values in parentheses; they're predeclared variables initialized to zero values.
- You can name only some returns by using _ for nameless ones.
- Named return names are local to the function and don't enforce names outside.
- Shadowing a named return value is a risk; named returns don't require you to use them.
- Named return parameters are essential for defer modifying return values (covered later).
- GOTCHA: Shadowing a named return value instead of assigning to it.

Blank Returns-Never Use These!

- RULE: A blank return (return with no values) returns the last values assigned to named return values.
- You still must put a return at the end; leaving it out is a compile-time error.
- Blank returns make data flow harder to understand; never use them.
- GOTCHA: Using blank returns (naked returns) in a function that returns values.

Functions Are Values

- RULE: Functions are values; the type is built from func plus parameter and return types (the signature).
- Declare a function variable: var myFuncVariable func(string) int.
- The zero value of a function variable is nil; running a nil function variable panics.
- GOTCHA: Calling a nil function variable (panics).

Function Type Declarations

- RULE: Declare a function type with type name func(paramTypes) returnType.
- Useful for documentation and referring to a signature multiple times.
- GOTCHA: none in text

Anonymous Functions

- RULE: Define a function inside a function with func immediately followed by parameters and return values (no name).
- It's a compile-time error to put a function name between func and the parameters.
- Call an anonymous function with parentheses; you can call it inline immediately.
- Useful for defer and launching goroutines.
- GOTCHA: Placing a name between func and the parameters (compile-time error).

Closures

- RULE: Functions declared inside functions are closures; they can access and modify outer-function variables.
- Shadowing inside a closure with := creates a new variable that ends when the closure exits.
- Use = to modify an outer variable inside a closure.
- GOTCHA: Using := instead of = inside a closure (creates a shadow, not a modification).

Passing Functions as Parameters

- RULE: Pass functions as parameters by specifying the function's parameter and return types.
- Useful for performing different operations on the same data (e.g., sort.Slice with a closure).

- GOTCHA: none in text

Returning Functions from Functions

- RULE: Return a closure from a function to carry state out (e.g., makeMult returns a multiplier).
- Closures let you use variables from your function outside of it.
- GOTCHA: none in text

defer

- RULE: defer delays a function/method/closure call until the surrounding function exits (resource cleanup).
- Multiple defers run LIFO (last registered runs first); defer code runs after the return statement.
- Defer arguments are evaluated immediately and stored.
- A deferred function can't read returned values unless you use named return values.
- New developers forget the parentheses after the deferred function (compile-time error if omitted).
- GOTCHA: Forgetting parentheses after the deferred function.

Go Is Call by Value

- RULE: Go always copies the value passed to a parameter; functions don't modify the caller's variables.
- Maps and slices are implemented with pointers, so their contents can be modified through a function.
- You can modify slice elements but not lengthen a slice passed to a function.
- Every type is a value type; sometimes the value is a pointer.
- Call by value makes data flow easier to understand; use a pointer when you need to pass something mutable.
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises test functions; solutions in the Chapter 5 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You've learned Go functions; next chapter covers pointers.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
func f(a, b int) int {}	Declare a function
func f(vals ...int) []int {}	Variadic parameter
return a, b, err	Return multiple values
result, _, err := f()	Ignore a return value with _
func f() (r int) {}	Named return value
return	Blank return (never use)
var f func(int) int {}	Function variable
defer f.Close()	Run cleanup on exit
f := func() {}	Anonymous function

SELF-TEST

1. What must you do when a Go function returns multiple values?

2. Which symbol do you use to ignore a return value you don't need?
3. What keyword delays a call until the surrounding function exits?
4. In what order do multiple defers run?
5. Are maps and slices modified through a function visible to the caller?

ANSWERS

1. Assign each returned value to a variable
2. Underscore (_)
3. defer
4. Last-in, first-out (LIFO)
5. Yes

CH06

Chapter 6: Pointers

SECTIONS COVERED

A Quick Pointer Primer

Don't Fear the Pointers

Pointers Indicate Mutable Parameters

Pointers Are a Last Resort

Pointer Passing Performance

The Zero Value Versus No Value

The Difference Between Maps and Slices

Slices as Buffers

Reducing the Garbage Collector's Workload

Tuning the Garbage Collector

Exercises

Wrapping Up

GOAL

Use Go pointers to mutate data, decide between value and pointer, and understand stack vs. heap and GC tuning.

KEY IDEAS

1. & takes an address; * dereferences; never dereference a nil pointer.

```
*pointerToX
```

2. new returns a pointer to a zero value; &struct{} creates a pointer to a struct literal.

```
var x = new(int)
```

3. Pass a pointer to let a function modify the original data.

```
update(&x)
```

4. Use pointers sparingly; favor values unless mutation or large data is needed.

```
x := &Foo{}
```

5. A slice holds length, capacity, and a pointer; contents are shared but length isn't.

```
[]int{1, 2, 3}
```

FULL NOTES

A Quick Pointer Primer

- RULE: A pointer holds the memory address where another variable is stored; & gets the address, * dereferences.
- The zero value for a pointer is nil; nil is untyped and not convertible to/from numbers.
- Dereferencing a nil pointer panics; check non-nil first.
- A pointer type is written * before a type: *int.
- new(int) returns a pointer to a zero-value int; new is rarely used.
- You can't take the address of a primitive literal or constant; use a variable or a helper.
- GOTCHA: Dereferencing a nil pointer (panics).

Don't Fear the Pointers

- RULE: Pointers are familiar class-like behavior; nonpointer structs are the unusual part of Go.
- Java/Python/JavaScript/Ruby class instances are implemented as pointers; they're pass-by-value of the pointer.
- Go gives you the choice of pointers or values for both primitives and structs.
- Most of the time, use a value; values make data flow clearer and reduce GC work.
- GOTCHA: none in text

Pointers Indicate Mutable Parameters

- RULE: Use a pointer parameter to indicate a parameter is mutable (the function can modify the original).
- Passing a pointer copies the pointer, which still points to the original data.
- You can't make a nil pointer non-nil inside a function (you only have a copy of the pointer).
- To change the value a pointer points to, dereference it: *px = 20.
- GOTCHA: Reassigning the pointer parameter instead of dereferencing (doesn't change the original).

Pointers Are a Last Resort

- RULE: Rather than passing a pointer to populate a struct, have the function instantiate and return the struct.
- Use pointer parameters to modify a variable mainly when the function expects an interface (e.g., json.Unmarshal).
- Favor value types as return types; use a pointer return only if internal state must be modified.
- GOTCHA: Passing pointers to populate structs instead of returning values.

Pointer Passing Performance

- RULE: Passing a pointer takes constant time regardless of data size; passing a value gets slower as data grows.
- Returning a pointer is slower than returning a value for small data; the flip happens at large sizes.
- The performance difference is tiny for most cases; consider pointers only when passing megabytes.

- GOTCHA: none in text

The Zero Value Versus No Value

- RULE: Use a nil pointer to distinguish an unassigned variable/field from one assigned the zero value.
- Rather than returning a nil pointer, return a value type and a boolean (comma ok idiom).
- For JSON/external protocols, use pointer fields for nullable values; otherwise prefer a value type plus a boolean.
- GOTCHA: Returning a nil pointer from a function (use value + bool instead).

The Difference Between Maps and Slices

- RULE: A map is implemented as a pointer to a struct; passing a map copies the pointer, so changes are reflected.
- A slice is a struct of length, capacity, and a pointer; copying a slice copies these three fields.
- Slice contents are shared between copy and original; changes to elements are visible in both.
- Appending changes length in the copy only if capacity allows; otherwise new memory is allocated.
- The original slice's length is unchanged by appends to a copy.
- You can pass a slice of any size to a function because the passed type is the same (length, capacity, pointer).
- GOTCHA: Assuming append on a slice copy changes the original slice's length.

Slices as Buffers

- RULE: Reuse a single byte slice as a buffer when reading from a file/network instead of allocating each time.
- You can change the buffer's contents up to its current length but not its length/capacity when passed to a function.
- GOTCHA: none in text

Reducing the Garbage Collector's Workload

- RULE: "Garbage" is data with no pointers pointing to it; the GC recovers it.
- Stack allocation is fast; heap allocation is managed by the GC and more expensive.
- Values with compile-time-known size can be stack-allocated; pointers to them are also on the stack.
- If a pointer to a local variable is returned or escapes, the data is stored on the heap (escape analysis).
- Go favors low-latency GC; each cycle pauses the program for under 500 microseconds.
- Slices of structs/primitives store data sequentially in memory (fast); slices of pointers scatter data (slow).
- Create less garbage; idiomatic Go is also the most efficient.
- GOTCHA: Returning a pointer to a local variable in C (Go handles it via heap allocation, but escape analysis is conservative).

Tuning the Garbage Collector

- RULE: GOGC controls when the next GC cycle triggers (default 100 = roughly double heap size).
- Lower GOGC = smaller target heap, more CPU; higher GOGC = larger heap, less CPU; doubling GOGC roughly halves GC CPU time.
- GOGC=off disables GC (faster but can exhaust memory in long-running processes).
- GOMEMLIMIT sets a maximum memory limit (in bytes; KiB/MiB/GiB/TiB suffixes); it's a soft limit.
- Use GOGC and GOMEMLIMIT together for a reasonable GC pace and a respected maximum.
- GOTCHA: Disabling GC in a long-running process (can use all available memory).

Exercises

- RULE: Chapter exercises reinforce pointers; answers in the Chapter 6 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You now understand pointers, when to use them, and memory/GC basics; next is methods, interfaces, types.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
&x	Address of x
*p	Dereference pointer p
var p *int	Declare a pointer to int
new(int)	Pointer to a zero-value int
&Foo{}	Pointer to a struct literal
update(&x)	Pass a pointer to mutate x
GOGC=100	GC target heap growth percent
GOMEMLIMIT=3GiB	Max memory limit

SELF-TEST

1. What does the & operator do?
2. What happens if you dereference a nil pointer?
3. Can you make a nil pointer non-nil inside a function?
4. What three fields make up a slice's internal struct?
5. What is the default value of GOGC?

ANSWERS

1. Returns the memory address of a value
2. The program panics
3. No
4. Length, capacity, and a pointer
5. 100

CH07

Chapter 7: Types, Methods, and Interfaces

SECTIONS COVERED

Types in Go

Methods

Pointer Receivers and Value Receivers

Code Your Methods for nil Instances

Methods Are Functions Too

Functions Versus Methods

Type Declarations Aren't Inheritance

Types Are Executable Documentation

iota Is for Enumerations-Sometimes

Use Embedding for Composition

Embedding Is Not Inheritance

A Quick Lesson on Interfaces

Interfaces Are Type-Safe Duck Typing

Embedding and Interfaces

Accept Interfaces, Return Structs

Interfaces and nil

Interfaces Are Comparable

The Empty Interface Says Nothing

Type Assertions and Type Switches

Use Type Assertions and Type Switches Sparingly

Function Types Are a Bridge to Interfaces

Implicit Interfaces Make Dependency Injection Easier

Wire

Go Isn't Particularly Object-Oriented (and That's Great)

Exercises

Wrapping Up

GOAL

Define types and methods, build and use implicit interfaces, and apply composition over inheritance.

KEY IDEAS

1. Interfaces are satisfied implicitly; a type implements one if its method set matches.

```
var w io.Writer = buf
```

2. Pointer receivers let methods mutate the receiver and handle nil instances.

```
func (c *Counter) Increment()
```

3. Embedding promotes fields and methods but is not inheritance.

```
type Manager struct { Employee }
```

4. Use type assertions with comma ok; type switches to extract concrete types.

```
i2, ok := i.(int)
```

5. Accept interfaces, return structs.

```
func process(r io.Reader) error
```

FULL NOTES

Types in Go

- RULE: Define a user-defined type with type Name underlying (e.g., type Score int).
- An abstract type specifies what; a concrete type specifies what and how.
- You can declare a type at any block level, but access it only within its scope.
- GOTCHA: none in text

Methods

- RULE: A method is a function with a receiver between func and the method name.
- The receiver name is conventionally a short abbreviation of the type; don't use this/self.
- Methods must be defined at package block level and in the same package as the type.
- Method names can't be overloaded; same name for different types is OK, not on the same type.
- GOTCHA: none in text

Pointer Receivers and Value Receivers

- RULE: Use a pointer receiver if the method modifies the receiver or must handle nil instances; otherwise use a value receiver.
- If a type has any pointer receiver methods, be consistent and use pointer receivers for all methods.
- Calling a pointer-receiver method on an addressable value auto-takes its address: c.Increment() becomes (&c).Increment().
- Calling a value-receiver method on a pointer auto-dereferences: c.String() becomes (*c).String().
- Calling a value-receiver method on a nil pointer compiles but panics at runtime.
- For a pointer instance, both pointer and value receiver methods are in the method set; for a value instance, only value receivers.
- GOTCHA: Calling a value-receiver method on a nil pointer (panics).

Code Your Methods for nil Instances

- RULE: A pointer-receiver method can handle a nil receiver; a value-receiver method panics on nil.
- Write nil-receiver methods when expecting nil makes code simpler (e.g., binary tree Insert/Contains).
- You can't write a pointer-receiver method that makes the original nil pointer non-nil.
- If nil is a fatal flaw, let it panic; if recoverable, check for nil and return an error.
- GOTCHA: none in text

Methods Are Functions Too

- RULE: Use a method anywhere a function type is expected (method value: f1 := myAdder.AddTo).
- A method expression uses the type itself: f2 := Adder.AddTo (signature func(Adder, int) int).
- Method values capture the instance's fields like a closure.
- GOTCHA: none in text

Functions Versus Methods

- RULE: Use a function if logic depends only on input parameters; use a method if it depends on configured/changed data stored in a struct.
- Package-level state should be effectively immutable.
- GOTCHA: none in text

Type Declarations Aren't Inheritance

- RULE: Declaring a type based on another (type HighScore Score) shares the underlying type but creates no hierarchy.
- You can't assign HighScore to Score without a conversion; methods on Score aren't on HighScore.
- A conversion between types with the same underlying type keeps storage but associates different

methods.

- GOTCHA: none in text

Types Are Executable Documentation

- RULE: User-defined types name a concept and describe expected data (e.g., Percentage vs int).
- When you have the same underlying data but different operations, make two types.
- GOTCHA: none in text

iota Is for Enumerations-Sometimes

- RULE: iota assigns increasing values starting at 0 in a const block; the type repeats on subsequent lines.
- Best practice: define a type based on int, then use iota in a const block for "internal" values referred to by name.
- Don't use iota when the actual value matters (spec/database values); specify them explicitly.
- Inserting a constant in the middle renumbers all subsequent ones.
- The zero value can be a sensible default (e.g., Uncategorized).
- GOTCHA: Using iota when the actual constant value matters.

Use Embedding for Composition

- RULE: Embed a field without a name to compose; its fields and methods are promoted to the containing struct.
- Promoted fields/methods can be invoked directly on the outer struct.
- If names collide, use the embedded type explicitly (o.Inner.X).
- GOTCHA: none in text

Embedding Is Not Inheritance

- RULE: You can't assign Manager to Employee; access the embedded field explicitly (m.Employee).
- Go has no dynamic dispatch for concrete types; embedded methods don't know they're embedded.
- Embedded methods still count toward the containing struct's method set (can satisfy interfaces).
- GOTCHA: Treating embedding like inheritance.

A Quick Lesson on Interfaces

- RULE: Declare an interface with type Name interface { method list }; it's the only abstract type in Go.
- Interfaces are usually named with "er" endings (io.Reader, io.Writer, http.Handler).
- The method set of a pointer includes pointer and value receivers; a value includes only value receivers.
- GOTCHA: none in text

Interfaces Are Type-Safe Duck Typing

- RULE: Interfaces are implicit; if a concrete type's method set contains all interface methods, it implements the interface.
- A concrete type never declares it implements an interface.
- The client code defines the interface to state what it needs; interfaces specify what callers need.
- It's fine for a type to have extra methods beyond the interface.
- GOTCHA: none in text

Embedding and Interfaces

- RULE: You can embed an interface in an interface (io.ReadCloser embeds io.Reader and io.Closer).
- You can embed an interface in a struct (used for stubs).
- GOTCHA: none in text

Accept Interfaces, Return Structs

- RULE: Accept interfaces in function parameters; return concrete types from functions.
- Returning concrete types lets you add methods/fields without breaking callers; adding an interface

method breaks all implementers.

- Return an interface when needed (e.g., error, database drivers); returning a struct avoids a heap allocation.
- GOTCHA: none in text

Interfaces and nil

- RULE: An interface is nil only if both its type and value fields are nil; a non-nil type means non-nil interface.
- A nil interface assigned a nil concrete instance is still non-nil (type field set).
- Calling methods on a nil interface panics.
- Use reflection to check if the value in a non-nil-type interface is nil.
- GOTCHA: Assuming an interface holding a nil pointer is nil (it isn't).

Interfaces Are Comparable

- RULE: Two interfaces are equal only if their types and values are equal; comparing incomparable types panics at runtime.
- A map can use an interface as a key, but an incomparable key panics when added.
- Be careful with ==/!= on interfaces and interface map keys.
- GOTCHA: Comparing interfaces wrapping incomparable types with == (panics).

The Empty Interface Says Nothing

- RULE: interface{} (alias any) stores a value of any type; use any for new code.
- any is useful for uncertain-schema external data (e.g., map[string]any for JSON).
- Avoid using any; Go is strongly typed and workarounds are unidiomatic.
- Pre-existing containers use empty interface; use generics for new ones.
- Read the value back with type assertions/type switches (or reflection).
- GOTCHA: none in text

Type Assertions and Type Switches

- RULE: i2 := i.(MyInt) asserts the concrete type; wrong assertions panic.
- Use the comma ok idiom: i2, ok := i.(int) to avoid panics.
- A type assertion reveals the stored type; a type conversion changes a value's type.
- Assertions are checked at runtime and can fail; use comma ok even when confident.
- A type switch: switch j := i.(type) { case int: ... } extracts the concrete type.
- Shadowing (i := i.(type)) is idiomatic in type switches.
- Include a default case to handle unknown future implementations.
- GOTCHA: Using a bare type assertion that can panic at runtime.

Use Type Assertions and Type Switches Sparingly

- RULE: Treat parameters/returns as the supplied type; don't dig for the concrete type unless necessary.
- Useful for optional interfaces (e.g., io.Copy detects io.WriterTo/io.ReaderFrom).
- Optional interfaces can't be detected through decorators/wrappers.
- For wrapped errors, use errors.Is and errors.As, not type switches.
- GOTCHA: none in text

Function Types Are a Bridge to Interfaces

- RULE: A user-defined function type can have methods, letting functions implement interfaces (e.g., http.HandlerFunc).
- Use a function-type parameter for simple cases (sort.Slice); use an interface when the function depends on other state.
- GOTCHA: none in text

Implicit Interfaces Make Dependency Injection Easier

- RULE: Define interfaces for dependencies; inject them via constructor functions returning structs.
- The main function is the only place that knows concrete types; swapping implementations changes only main.
- Dependency injection makes testing easier.
- GOTCHA: none in text

Wire

- RULE: Wire is a Google dependency injection helper that uses code generation to create the concrete declarations you'd write by hand.
- GOTCHA: none in text

Go Isn't Particularly Object-Oriented (and That's Great)

- RULE: Go has no inheritance, method overriding, or objects; it's practical, simple, readable, and maintainable.
- Borrow concepts from many paradigms but write idiomatic Go, not shoehorned code.
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises build a basketball league with Team, League, Ranker, and RankPrinter; answers in Chapter 7 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You learned types, methods, interfaces, and best practices; next is generics.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
type Name struct { }	Define a struct type
func (r T) Method() {}	Method with receiver r
type I interface { M() }	Define an interface
var v I = concrete	Implicit interface satisfaction
type Manager struct { Employee }	Embed Employee in Manager
i2, ok := i.(int)	Safe type assertion
switch j := i.(type) {}	Type switch
func process(r io.Reader) {}	Accept interface, return struct

SELF-TEST

1. Can a concrete type declare that it implements an interface?
2. Which receiver must a method use to modify its receiver?
3. What happens when you call a value-receiver method on a nil pointer?
4. Is embedding inheritance?
5. What does iota start counting from?

ANSWERS

1. No; it's implicit
2. Pointer receiver
3. It panics at runtime
4. No
5. 0

CH08

Chapter 8: Generics

SECTIONS COVERED

Generics Reduce Repetitive Code and Increase Type Safety

Introducing Generics in Go

Generic Functions Abstract Algorithms

Generics and Interfaces

Use Type Terms to Specify Operators

Type Inference and Generics

Type Elements Limit Constants

Combining Generic Functions with Generic Data Structures

More on comparable

Things That Are Left Out

Idiomatic Go and Generics

Adding Generics to the Standard Library

Future Features Unlocked

Exercises

Wrapping Up

GOAL

Write type-safe functions and data structures that work with multiple types using type parameters.

KEY IDEAS

1. A type parameter is written [T constraint] after a type or function name.

```
type Stack[T any] struct
```

2. The comparable constraint lets you use == on a type parameter.

```
type Stack[T comparable] struct
```

3. Type terms (~int) match a type and all types with that underlying type.

```
type Integer interface { ~int | ~int64 }
```

4. Type inference fills in type arguments when possible; otherwise specify them.

```
Convert[int, int64](a)
```

5. Generics add compile-time type safety where interfaces with any would panic at runtime.

```
Stack[int]
```

FULL NOTES

Generics Reduce Repetitive Code and Increase Type Safety

- RULE: Generics (type parameters) let you write functions and structs that work with multiple concrete types while keeping compile-time type safety.
- Before generics, reusing logic across types meant duplication or using any/interfaces, which loses compile-time checks and can panic at runtime.
- GOTCHA: none in text

Introducing Generics in Go

- RULE: Write [T constraint] after the type name; inside, use T as a type; refer to the generic type as Stack[T] in receivers.
- any means any type is usable; comparable means types usable with == and !=.
- You can't return nil for a generic value type; get a zero value with var zero T.
- Using a generic type: var s Stack[int]; the compiler rejects wrong types (e.g., pushing a string).
- GOTCHA: Returning nil for a generic type (not valid for value types).

Generic Functions Abstract Algorithms

- RULE: Generic functions place type parameters after the function name: func Map[T1, T2 any](s []T1, f func(T1) T2) []T2.
- Map, Reduce, and Filter show how generics replace per-type implementations.
- GOTCHA: none in text

Generics and Interfaces

- RULE: Any interface can be a type constraint, not just any and comparable.
- You can create interfaces with type parameters (e.g., Differ[T any]).
- Pair[T fmt.Stringer] requires fields of a type meeting fmt.Stringer.
- GOTCHA: none in text

Use Type Terms to Specify Operators

- RULE: Type terms list concrete types separated by | inside an interface used as a constraint (e.g., int | int8 | ...).
- Operators valid for all listed types are supported; % is valid only for integers.
- Interfaces with type elements are valid only as constraints; using one as a variable/field/return/parameter type is a compile-time error.
- By default type terms match exactly; use ~ before a term (e.g., ~int) to match any type with that underlying type.
- Ordered (~int | ~uint | ... | ~string) supports ==, !=, <, >, <=, >=; cmp package provides Ordered, Compare, and Less (Go 1.21).
- You can combine type elements and method elements (e.g., ~int plus String() string).
- An impossible constraint (int with no methods) can't be instantiated; using it causes a compiler error.
- GOTCHA: Using a type-element interface as a variable type (compile-time error).

Type Inference and Generics

- RULE: Go infers type arguments for generic calls when possible (e.g., Map, Filter, Reduce).
- When a type parameter is used only as a return value, inference fails; specify all type arguments: Convert[int, int64](a).
- GOTCHA: none in text

Type Elements Limit Constants

- RULE: Type terms also limit which constants can be assigned; a constant must be valid for all listed types.
- No constant is valid for all types in Ordered, so you can't assign one to a variable of that generic type.
- PlusOneThousand[T Integer] won't compile (1000 doesn't fit int8); PlusOneHundred does.
- GOTCHA: none in text

Combining Generic Functions with Generic Data Structures

- RULE: Combine a generic ordering function with a generic tree (OrderableFunc[T] func(t1, t2 T) int) to build a type-safe tree for any type.
- Use cmp.Compare[int] or a custom order function/method expression (Person.Order) with NewTree.
- GOTCHA: none in text

More on comparable

- RULE: comparable works with generics, but an interface wrapping an incomparable type still panics at runtime when compared with ==.
- Comparer[T comparable] compiles for int and ThingerInt; ThingerSlice fails at compile time.
- Assigning a ThingerSlice to a Thinger variable then comparing panics at runtime (comparing uncomparable type).
- GOTCHA: Comparing interface variables wrapping incomparable types with == (panics).

Things That Are Left Out

- RULE: Go generics lack operator overloading, function/method overloading, parameterized methods, variadic type parameters, specialization, currying, and metaprogramming.
- No range over user-defined containers or [] indexing on them.
- GOTCHA: none in text

Idiomatic Go and Generics

- RULE: Use any instead of interface{} for unspecified types; handle different slice types with one function.
- Don't rush to convert all code to generics; old code still works.
- Don't convert interface parameters to generic type parameters hoping for performance (can be ~30% slower for trivial functions).
- Measure with benchmarks/profiling; write maintainable programs that are fast enough.
- GOTCHA: none in text

Adding Generics to the Standard Library

- RULE: Go 1.18 added any and comparable; Go 1.21+ added generic functions in slices and maps packages (Equal, EqualFunc, Insert, Delete, DeleteFunc, maps.Clone).
- sync.OnceValue and sync.OnceValues use generics.
- Prefer standard library generic functions over writing your own.
- GOTCHA: none in text

Future Features Unlocked

- RULE: Generics may enable future features like sum types (a bounded set of types for a field).
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises apply generics; solutions in Chapter 8 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You learned generics; next chapter covers errors.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
[T any]	Type parameter accepting any type
[T comparable]	Type parameter usable with ==
~int	Type term matching int and its underlying types
type Stack[T any] struct	Generic struct
func Map[T1, T2 any](...)	Generic function
cmp.Compare	Compare two orderable values
any	Alias for interface{}

SELF-TEST

1. What is the difference between any and comparable as constraints?
2. What does the ~ symbol do before a type term?
3. When must you specify type arguments explicitly instead of relying on inference?
4. Which interface in the cmp package supports ordering operators?
5. Can you add methods to a generic type's methods with their own type parameters?

ANSWERS

1. any accepts any type; comparable only types usable with ==
2. Matches any type with that underlying type
3. When a type parameter is used only as a return value
4. Ordered
5. No

CH09

Chapter 9: Errors

SECTIONS COVERED

How to Handle Errors: The Basics

Use Strings for Simple Errors

Sentinel Errors

Errors Are Values

Wrapping Errors

Wrapping Multiple Errors

Is and As

Wrapping Errors with defer

panic and recover

Getting a Stack Trace from an Error

Exercises

Wrapping Up

GOAL

Handle errors by returning and checking them, wrap and inspect errors, and use panic/recover only for fatal situations.

KEY IDEAS

1. Return error as the last value; check it with if err != nil; return nil on success.

`if err != nil { return err }`

2. Create simple errors with errors.New or fmt.Errorf.

`errors.New("message")`

3. Wrap errors with %w and find them with errors.Is / errors.As.

`fmt.Errorf("in f: %w", err)`

4. Sentinel errors are package-level variables starting with Err; compare with == or errors.Is.

`err == zip.ErrFormat`

5. panic and recover are for fatal situations, not ordinary error handling.

`defer func() { recover() }()`

FULL NOTES

How to Handle Errors: The Basics

- RULE: Go returns a value of type error as the last return value; nil means success, a non-nil error means failure; check it with an if statement.
- Create a new error from a string with errors.New; error messages aren't capitalized and don't end with punctuation or a newline.
- Set other return values to zero values when returning a non-nil error (exception: sentinel errors).
- error is a built-in interface with a single method: Error() string.
- Go uses returned errors (not exceptions) because exceptions add unclear code paths and because the compiler requires variables to be read, forcing you to handle or explicitly ignore errors.
- GOTCHA: none in text

Use Strings for Simple Errors

- RULE: errors.New takes a string and returns an error; fmt.Errorf lets you format runtime info into the error

message using Printf verbs.

- Passing an error to fmt.Println calls its Error method automatically.
- GOTCHA: none in text

Sentinel Errors

- RULE: Sentinel errors are specific error values signifying a state where processing can't continue; declared at package level, names start with Err (except io.EOF), treated as read-only.
- Test with == when a function's documentation says it returns a sentinel error.
- Define a sentinel error only if you truly need one; it becomes part of your public API forever.
- Reuse standard library sentinel errors where possible.
- GOTCHA: none in text

Errors Are Values

- RULE: error is an interface, so you can define custom error types that carry extra information (e.g., a status code).
- Always use error as the return type so callers don't depend on the specific error type.
- Never return an uninitialized custom error instance; an interface holding a non-nil type is non-nil even if the value is nil.
- Fix: explicitly return nil on success, or declare the local variable as type error.
- Don't use type assertions/switches on custom errors; use errors.As.
- GOTCHA: Returning a non-nil interface holding a nil custom error value (appears non-nil).

Wrapping Errors

- RULE: Wrapping preserves an error while adding info (e.g., the function that received it); a series of wrapped errors is an error tree.
- fmt.Errorf with the %w verb wraps an error; write ": %w" at the end of the format string with the wrapped error as the last parameter.
- errors.Unwrap returns the wrapped error or nil (don't call it directly; use errors.Is/errors.As).
- To wrap with a custom error type, implement Unwrap() error.
- Not all errors need wrapping; sometimes return a brand-new error.
- To include a message without wrapping, use %v instead of %w.
- GOTCHA: none in text

Wrapping Multiple Errors

- RULE: errors.Join merges multiple errors into a single error.
- You can pass multiple %w verbs to fmt.Errorf to wrap several errors.
- For multiple wrapped errors, implement Unwrap() []error (errors.Unwrap returns nil for these).
- GOTCHA: none in text

Is and As

- RULE: errors.Is checks whether an error or any wrapped error matches a sentinel; it walks the error tree.
- errors.Is uses == by default; implement an Is(target error) method on your error for custom matching.
- Use errors.Is to find a specific instance/values; use errors.As to find a specific type.
- errors.As takes the error and a pointer to a variable of the type you're looking for (or a pointer to an interface).
- Passing a non-pointer, or a pointer to a non-error/non-interface, to errors.As panics.
- You can override errors.As with an As method (nontrivial, needs reflection).
- GOTCHA: Passing the wrong second argument to errors.As (panics).

Wrapping Errors with defer

- RULE: Use defer with named return values to wrap every error with the same message (check err in the

closure and reassign a wrapped error).

- If you name one return value, name them all; use _ for unused ones.
- GOTCHA: none in text

panic and recover

- RULE: A panic is a runtime state when Go can't figure out what to do next (almost always a programming error); it runs defers and exits with a stack trace.
- Create your own panics with the built-in panic(any).
- recover is called from within a defer; it returns the panic value or nil and lets execution continue.
- Reserve panics for fatal situations; recover as a graceful shutdown; rarely continue after a panic.
- If a panic is possible in a library's public API, use recover to convert it into an error and return it.
- panic(nil) before Go 1.21 was silent; Go 1.21+ treats it as panic(new(runtime.PanicNilError)).
- GOTCHA: Trying to continue normal execution after a panic.

Getting a Stack Trace from an Error

- RULE: Go doesn't print stack traces by default; use fmt.Printf with %+v for verbose output.
- Use -trimpath when building to replace full file paths with package names.
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises improve error handling in sample code; solutions in Chapter 9 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You learned how to define and examine errors, plus panic and recover; next is packages and modules.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
errors.New("msg")	Create a simple error
fmt.Errorf("w: %w", err)	Wrap an error
if err != nil {}	Check a returned error
errors.Is(err, target)	Check error tree for a sentinel
errors.As(err, &target)	Find error of a specific type
errors.Join(errs...)	Merge multiple errors
defer f.Close()	Run cleanup on exit
panic(msg)	Trigger a panic
recover()	Catch a panic (in defer)

SELF-TEST

1. Where should the error return value be positioned in a function's return values?
2. Which fmt.Errorf verb wraps an error instead of just formatting it?
3. Which function checks whether an error or any wrapped error matches a sentinel?
4. Which function finds an error of a specific type in an error tree?

5. From where must recover be called?

ANSWERS

1. Last
2. %w
3. errors.Is
4. errors.As
5. From within a defer

CH10

Chapter 10: Modules, Packages, and Imports

SECTIONS COVERED

Repositories, Modules, and Packages

Using go.mod

Use the go Directive to Manage Go Build Versions

The require Directive

Building Packages

Importing and Exporting

Creating and Accessing a Package

Naming Packages

Overriding a Package's Name

Documenting Your Code with Go Doc Comments

Using the internal Package

Avoiding Circular Dependencies

Organizing Your Module

Gracefully Renaming and Reorganizing Your API

Avoiding the init Function if Possible

Working with Modules

Importing Third-Party Code

Working with Versions

Minimal Version Selection

Updating to Compatible Versions

Updating to Incompatible Versions

Vendoring

Using pkg.go.dev

Publishing Your Module
Versioning Your Module
Overriding Dependencies
Retracting a Version of Your Module
Using Workspaces to Modify Modules Simultaneously
Module Proxy Servers
Specifying a Proxy Server
Using Private Repositories
Additional Details
Exercises
Wrapping Up

GOAL

Organize code into packages and modules, manage dependencies with go.mod, and publish your own module.

KEY IDEAS

1. Capitalized identifiers are exported; lowercase ones are package-private.

```
func Double
```

2. go.mod starts with module and go directives; require lists dependencies.

```
module example.com/m
```

3. go get adds dependencies; go mod tidy synchronizes go.mod/go.sum.

```
go get ./...
```

4. Minimal version selection picks the lowest version satisfying all requirements.
5. Semantic import versioning: major versions >1 must end the module path in /vN.

```
example.com/m/v2
```

FULL NOTES

Repositories, Modules, and Packages

- RULE: A repository holds source; a module is versioned/distributed as one unit; a module contains packages (directories of source).
- Don't store more than one module per repository (discouraged); everything in a module is versioned together.
- A module path is a globally unique identifier, usually based on the repository URL; case-sensitive; avoid uppercase.
- GOTCHA: none in text

Using go.mod

- RULE: A directory becomes a module when it has a valid go.mod; use go mod init MODULE_PATH to create it.
- go.mod starts with module and go directives; go specifies the minimum compatible Go version.
- Source code must be compatible with the specified Go version (e.g., go 1.12 disallows numeric literal underscores added in 1.13).
- GOTCHA: none in text

Use the go Directive to Manage Go Build Versions

- RULE: If go directive is newer than installed Go: 1.20 or earlier ignores it; 1.21+ downloads the newer version (default).
- Control with toolchain directive and GOTOOLCHAIN: auto (default), local, or a specific version (go1.20.4).
- GOTOOLCHAIN overrides toolchain directive if both set.
- Go 1.22+ with go directive >=1.22: for loop creates new index/value per iteration (per module, backward-breaking).
- GOTCHA: none in text

The require Directive

- RULE: require directives list modules your module depends on and minimum versions; first section = direct deps, second = indirect deps.
- Indirect is just documentation; no functional difference except one go get situation.
- GOTCHA: none in text

Building Packages

- RULE: The package clause is the first nonblank noncomment line; every file in a directory must have the same package clause.
- Import path = module path + path to package within the module (standard library needs only the package name).
- Importing an unused package is a compile-time error.
- Package name comes from the package clause, not the directory name; make them match when possible.
- Use package main for application entry points (can't be imported).
- GOTCHA: none in text

Importing and Exporting

- RULE: Uppercase first letter = exported (visible outside package); lowercase or underscore = package-private.
- Exported identifiers are part of your package's API; document them and keep backward compatible.
- GOTCHA: none in text

Creating and Accessing a Package

- RULE: Import packages by path; call exported symbols with the package prefix (math.Double(2)).
- GOTCHA: none in text

Naming Packages

- RULE: Package names should be descriptive nouns; avoid util; don't repeat package name in function names (except when identifier == package name, like sort.Sort).
- Two functions can share a name if disambiguated by package (extract.Names vs format.Names).
- GOTCHA: none in text

Overriding a Package's Name

- RULE: Give an imported package an alternate name on collision: crand "crypto/rand".
- Dot import (. "pkg") puts identifiers in current namespace (discouraged); underscore (_ "pkg") triggers init without access (blank import).
- Package names can be shadowed; override to resolve conflicts.
- GOTCHA: none in text

Documenting Your Code with Go Doc Comments

- RULE: Comments directly before an item, no blank line; // prefix; first word is the symbol name; use

pkg.go.dev-compatible formatting.

- Package-level comments go before package clause; long ones in doc.go.
- At minimum, comment every exported identifier.
- go doc PACKAGE_NAME and go doc PACKAGE.IDENTIFIER display docs; pkgsite previews HTML locally.
- GOTCHA: none in text

Using the internal Package

- RULE: A package named internal is accessible only to its direct parent and sibling packages.
- External access is a compile-time error.
- GOTCHA: none in text

Avoiding Circular Dependencies

- RULE: Go forbids circular package dependencies (A imports B and B imports A).
- Fix by merging packages or moving the cycle-causing items to one package or a new package.
- GOTCHA: none in text

Organizing Your Module

- RULE: Small modules: single package. Applications: root is main, logic in internal. Libraries: root package matches repo name; apps under cmd/.
- Organize by slices of functionality to limit inter-package dependencies.
- Use internal for module-only symbols (Hyrum's law: all observable behaviors get depended on).
- Don't follow golang-standards repo layout (Go team calls it an antipattern).
- GOTCHA: none in text

Gracefully Renaming and Reorganizing Your API

- RULE: Don't remove original identifiers; provide alternates (wrapper functions, new constants, type aliases).
- A type alias (type Bar = Foo) is just another name; same fields/methods; assignable without conversion.
- Aliases can't give alternate names to package-level variables or struct fields.
- GOTCHA: none in text

Avoiding the init Function if Possible

- RULE: init runs when a package is first referenced; multiple inits allowed; avoid them due to unclear order and side effects.
- init is used for driver registration and initializing package-level variables that can't be set in one assignment.
- Package-level variables set via init should be effectively immutable.
- Blank imports (_ "pkg") trigger init but give no access to identifiers (considered obsolete for your own code).
- Document init behavior in a package comment.
- GOTCHA: none in text

Working with Modules

- RULE: This section introduces integrating with third-party modules and publishing your own.
- GOTCHA: none in text

Importing Third-Party Code

- RULE: Go builds applications from source into a single binary, including all dependencies.
- Import third-party packages by repository location; access exported items normally.
- go get downloads modules and updates go.mod; go.sum records hashes for repeatable builds.

- Always commit go.mod and go.sum.
- The module cache stores downloaded versions; go clean -modcache removes it.
- GOTCHA: none in text

Working with Versions

- RULE: Go picks the latest dependency version by default; specify an earlier version with go get module@v1.0.0.
- Use go list -m -versions module to see available versions.
- GOTCHA: none in text

Minimal Version Selection

- RULE: Go picks the lowest version of a dependency that satisfies all go.mod requirements across all modules.
- go mod graph shows the dependency graph.
- Go won't include multiple versions of the same package (unlike npm).
- GOTCHA: none in text

Updating to Compatible Versions

- RULE: go get -u=patch upgrades to the latest patch of the current minor; go get module@v1.1.0 then -u=patch; go get -u gets the latest.
- GOTCHA: none in text

Updating to Incompatible Versions

- RULE: Semantic import versioning: major version incremented; for major versions besides 0 and 1, the module path ends in /vN.
- Store v2 in a subdirectory named v2 or on a branch named vN; update import paths and go.mod module path to end in /vN.
- Tag new major versions as vN.0.0.
- GOTCHA: none in text

Vendoring

- RULE: go mod vendor creates a vendor directory with dependency copies; used instead of the module cache.
- Re-run go mod vendor after go get changes; builds refuse if vendor is out of date.
- Vendoring increases repo size but helps CI/CD with ephemeral build servers.
- GOTCHA: none in text

Using pkg.go.dev

- RULE: pkg.go.dev indexes Go modules and publishes godocs, license, README, dependencies, and dependents.
- GOTCHA: none in text

Publishing Your Module

- RULE: Publish by putting the module in a version control system (Git, Mercurial, etc.); no central upload needed.
- Check in go.mod and go.sum; include a LICENSE file; Go community favors permissive licenses (BSD, MIT, Apache).
- Don't write your own license.
- GOTCHA: none in text

Versioning Your Module

- RULE: Tag changes with semantic versioning; pre-releases use tags like v1.4.0-beta1 (must be specified

explicitly in go get).

- Backward-breaking changes need a new import path ending in /vN (subdirectory or branch).
- GOTCHA: none in text

Overriding Dependencies

- RULE: replace redirects all references to a module to a fork (replace old => new v1.0.0); can point to a local path (version omitted).
- Avoid local replace directives (won't build for others); use workspaces instead.
- exclude blocks a specific module version from being used.
- GOTCHA: none in text

Retracting a Version of Your Module

- RULE: retract in go.mod marks versions no one should use; can retract single versions or ranges ([v1.7.0, v1.8.5]); add a reason comment.
- Retracted versions keep working for existing builds but won't be matched by @latest or go get upgrades.
- Retract prevents others from using your versions; exclude blocks you from using another's versions.
- GOTCHA: none in text

Using Workspaces to Modify Modules Simultaneously

- RULE: Workspaces (go.work) let multiple local modules resolve to local source instead of remote versions.
- go work init and go work use create go.work (don't commit it).
- With a workspace, local changes are used even after a public require exists (set GOWORK=off to validate against public).
- Release by committing modules in dependency order and tagging.
- GOTCHA: Don't commit go.work to source control.

Module Proxy Servers

- RULE: go get uses a proxy server (Google's) by default; it caches modules and versions; a checksum database verifies module integrity.
- If hashes don't match, the module isn't installed.
- GOTCHA: none in text

Specifying a Proxy Server

- RULE: GOPROXY=direct disables proxying (download directly from repos); run your own proxy (Artifactory, Sonatype, Athens) and point GOPROXY to it.
- GOTCHA: none in text

Using Private Repositories

- RULE: Google's proxy can't fetch private modules; Go falls back to direct access, but this may leak repo names.
- A private proxy avoids leaking auth info in CI/CD and speeds up downloads.
- GOPRIVATE lists private repos (comma-separated) to download directly even with a public proxy.
- GOTCHA: none in text

Additional Details

- RULE: The Go Modules Reference covers more (other VCS, module cache structure, env vars, REST API).
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises build and version a module with an Add function; solutions in Chapter 10

repository.

- GOTCHA: none in text

Wrapping Up

- RULE: You learned modules, packages, third-party code, and publishing; next is Go tooling.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
go mod init path	Create a module and go.mod
go get ./...	Add dependencies from imports
go mod tidy	Sync go.mod/go.sum with source
import "path/pkg"	Import a package
func Double() {}	Exported (uppercase)
replace old => new	Redirect a module to a fork
go work use ./path	Add a module to a workspace
retract v1.5.0	Mark a version as retracted

SELF-TEST

1. What determines if a Go identifier is exported?
2. Which command downloads dependencies and updates go.mod?
3. Which command synchronizes go.mod and go.sum with source code?
4. What must a major version's module path end in (for versions > 1)?
5. Where must a go.work file NOT be committed?

ANSWERS

1. Whether its name starts with an uppercase letter
2. go get
3. go mod tidy
4. /vN (e.g., /v2)
5. Source control

CH11

Chapter 11: Go Tooling

SECTIONS COVERED

Using go run to Try Out Small Programs
Adding Third-Party Tools with go install
Improving Import Formatting with goimports
Using Code-Quality Scanners
staticcheck
revive
golangci-lint
Using govulncheck to Scan for Vulnerable Dependencies
Embedding Content into Your Program
Embedding Hidden Files
Using go generate
Working with go generate and Makefiles
Reading the Build Info Inside a Go Binary
Building Go Binaries for Other Platforms
Using Build Tags
Testing Versions of Go
Using go help to Learn More About Go Tooling
Exercises
Wrapping Up

GOAL

Run, install, format, lint, embed, cross-compile, and inspect Go programs with Go tools and third-party scanners.

KEY IDEAS

1. go run builds and executes a program in one step from a temp directory.

```
go run hello.go
```

2. go install downloads, compiles, and installs a tool; always use @version/@latest.

```
go install tool@latest
```

3. staticcheck, revive, and golangci-lint scan for bugs and nonidiomatic code.

```
staticcheck ./...
```

4. //go:embed embeds file contents into package-level string, []byte, or embed.FS variables.

```
//go:embed file.txt
```

5. Cross-compile with GOOS and GOARCH; build tags select code per platform.

```
GOOS=linux GOARCH=amd64 go build
```

FULL NOTES

Using go run to Try Out Small Programs

- RULE: go run compiles and executes in one step; the binary goes to a temp directory and is deleted after the program finishes.
- Use go run to treat a Go program like a script for rapid feedback.
- GOTCHA: none in text

Adding Third-Party Tools with go install

- RULE: go install downloads, compiles, and installs a tool from a module path; always add @version or @latest.
- Without @version/@latest, you may get an error or install the go.mod version (confusing behavior).
- Binaries go to $HOME/go/bin by default; set GOBIN to change it and add the directory to PATH.
- Don't set GOROOT or GOPATH; modules supersede GOPATH.
- Use go help environment for the full list of recognized environment variables.
- GOTCHA: Running go install without @version/@latest.

Improving Import Formatting with goimports

- RULE: goimports is an enhanced go fmt that also sorts imports, removes unused ones, and guesses missing imports (guesses can be inaccurate).
- Install with go install golang.org/x/tools/cmd/goimports@latest; run goimports -l -w .
- golang.org/x packages are outside the main Go tree with looser compatibility.
- GOTCHA: none in text

Using Code-Quality Scanners

- RULE: Linters check style and bugs go vet misses; they can have false positives/negatives-trust but verify.
- Each linter supports a comment to block errant results; explain why in the comment.
- GOTCHA: none in text

staticcheck

- RULE: staticcheck is the recommended third-party scanner (150+ checks, few false positives); install honnef.co/go/tools/cmd/staticcheck@latest and run staticcheck ./....
- Use staticcheck -explain <code> for an explanation of a finding.
- It catches things go vet misses, like unnecessary fmt.Sprintf and unused err values.
- GOTCHA: none in text

revive

- RULE: revive is based on golint; enables golint rules by default; config file enables more rules.
- Install github.com/mgechev/revive@latest; run with -config.
- GOTCHA: none in text

golangci-lint

- RULE: golangci-lint runs 50+ code-quality tools (go vet, staticcheck, revive, etc.); download a binary version; run golangci-lint run.
- Configure via .golangci.yml at the module root and commit it; don't use a home-directory config when working with others.
- Start with go vet, add staticcheck, then revive, then golangci-lint.
- GOTCHA: none in text

Using govulncheck to Scan for Vulnerable Dependencies

- RULE: govulncheck scans dependencies for known vulnerabilities in the standard library and third-party libraries (data from vuln.go.dev).
- Install golang.org/x/vuln/cmd/govulncheck@latest; run govulncheck ./....
- It gives the exact line calling vulnerable code; update to the fixed patch version.
- Use the smallest possible change when updating dependencies.
- govulncheck can scan binaries too (govulncheck -mode binary) using build info.
- GOTCHA: none in text

Embedding Content into Your Program

- RULE: Use //go:embed comments to embed file contents into package-level variables of type string, []byte, or embed.FS.
- Import _ "embed" to enable embedding for a single string/[]byte; embed.FS doesn't need a blank import.
- The comment must start with go:embed (no space after slashes) and be directly before the variable.
- Treat embedded variables as immutable; embedding is compiler-checked (invalid patterns fail compilation).
- GOTCHA: none in text

Embedding Hidden Files

- RULE: Files/dirs starting with . or _ aren't embedded by default; use dir/* to include hidden files in the root only, or all:dir for all subdirectories.
- Wildcards and ranges follow path.Match conventions (* matches 0+ chars, ? matches one).
- GOTCHA: none in text

Using go generate

- RULE: go generate runs commands found in //go:generate comments; it does nothing by itself but is used to generate source code (protobufs, stringer).
- Install protoc and protoc-gen-go for protobufs; install stringer for enum String methods.
- Commit generated source code to version control so others don't need the generator tools.
- Add a generate step to your Makefile as a dependency of build.
- GOTCHA: none in text

Working with go generate and Makefiles

- RULE: Use go generate to mechanically create source code; use the Makefile to validate and compile.
- Commit generated code; it lets others build without generator tools installed.
- Don't commit generated code if output varies per run (timestamps) or generation is very slow.
- GOTCHA: none in text

Reading the Build Info Inside a Go Binary

- RULE: Every go build binary contains build info (module versions, build commands, VCS revision).
- View it with go version -m binary; debug/buildinfo package reads it programmatically.
- govulncheck -mode binary scans binaries for vulnerable libraries.
- GOTCHA: none in text

Building Go Binaries for Other Platforms

- RULE: Cross-compile with GOOS (target OS) and GOARCH (target CPU); defaults to your current computer.
- Example: GOOS=linux GOARCH=amd64 go build.
- GOTCHA: none in text

Using Build Tags

- RULE: Filenames with _GOOS_GOARCH before .go restrict compilation to that platform (e.g., something_windows.go).
- Build tags are //go:build magic comments before the package clause, using ||, &&, !, and parentheses.
- Meta constraints: unix, cgo; custom tags controlled with -tags flag; //go:build ignore skips a file.
- No whitespace between // and go:build.
- GOTCHA: Whitespace between // and go:build (not recognized as a build tag).

Testing Versions of Go

- RULE: Install secondary Go versions with go install golang.org/dl/go1.19.2@latest then go1.19.2

download; use go1.19.2 instead of go.

- Secondary versions live in ~/sdk; uninstall by deleting from sdk and go/bin.
- GOTCHA: none in text

Using go help to Learn More About Go Tooling

- RULE: go help contains exhaustive info on commands, modules, import paths, nonpublic source code; e.g., go help importpath.
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises cover embedding, staticcheck, and cross-compilation; solutions in Chapter 11 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You learned Go tooling; next is concurrency.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
go run file.go	Build and run in one step
go install tool@latest	Install a third-party tool
staticcheck ./...	Scan module for code-quality issues
//go:embed file.txt	Embed file into a variable
GOOS=linux GOARCH=amd64 go build	Cross-compile
//go:build linux	Build tag for platform selection
go version -m binary	Show build info in a binary
govulncheck ./...	Scan for vulnerable dependencies

SELF-TEST

1. What must you always include when using go install?
2. Where do go install binaries go by default?
3. Which scanner is recommended as the one third-party scanner to use?
4. Which directive embeds a file into a Go binary?
5. Which environment variables control cross-compilation?

ANSWERS

1. @version or @latest
2. $HOME/go/bin
3. staticcheck
4. //go:embed
5. GOOS and GOARCH

CH12

Chapter 12: Concurrency in Go

SECTIONS COVERED

When to Use Concurrency

Goroutines

Channels

Reading, Writing, and Buffering

Using for-range and Channels

Closing a Channel

Understanding How Channels Behave

select

Concurrency Practices and Patterns

Keep Your APIs Concurrency-Free

Goroutines, for Loops, and Varying Variables

Always Clean Up Your Goroutines

Use the Context to Terminate Goroutines

Know When to Use Buffered and Unbuffered Channels

Implement Backpressure

Turn Off a case in a select

Time Out Code

Use WaitGroups

Run Code Exactly Once

Put Your Concurrent Tools Together

When to Use Mutexes Instead of Channels

Atomics-You Probably Don't Need These

Where to Learn More About Concurrency

Exercises

Wrapping Up

GOAL

Launch goroutines, communicate with channels and select, and clean up concurrency safely.

KEY IDEAS

1. Concurrency is not parallelism; it's about structuring independent components.
2. A goroutine is a lightweight thread managed by the Go runtime; launch with go.

```
go func() { }()
```

3. Channels connect goroutines; unbuffered by default; sender closes the channel.

```
ch := make(chan int)
```

4. select picks at random from ready cases; avoids deadlock from lock ordering.

```
select { case <-ch: }
```

5. Use context.WithCancel/WithTimeout and sync.WaitGroup to stop and wait for goroutines.

```
defer wg.Done()
```

FULL NOTES

When to Use Concurrency

- RULE: Use concurrency to structure independent components, not to blindly speed things up; concurrency is not parallelism.
- More concurrency doesn't mean more speed (Amdahl's law); overhead can exceed savings for fast in-memory work.
- Good for I/O (disk/network is thousands of times slower than in-memory); write serially first, then benchmark.
- GOTCHA: Putting everything in goroutines without measuring.

Goroutines

- RULE: A goroutine is a lightweight thread managed by the Go runtime; launch one with go before a function call.
- Goroutines are cheaper than OS threads, have small growable stacks, and switch without OS calls.
- Return values from a goroutine-launched function are ignored.
- GOTCHA: none in text

Channels

- RULE: Channels are built-in, created with make(chan T); they're reference types (passing passes a pointer); zero value is nil.
- <-ch reads; ch <- v writes; each value is read by only one goroutine.
- Use <-chan T (read-only) or chan<- T (write-only) in signatures so the compiler enforces direction.
- GOTCHA: none in text

Reading, Writing, and Buffering

- RULE: Unbuffered channels block the writer until a reader is ready and vice versa; they need two goroutines to operate.
- Buffered channels (make(chan T, n)) buffer limited writes; len shows buffered count, cap shows max size (can't change).
- Most of the time, use unbuffered channels.
- GOTCHA: none in text

Using for-range and Channels

- RULE: for v := range ch reads until the channel is closed; pauses when no value is available.
- A channel read loop has one variable (the value).
- GOTCHA: none in text

Closing a Channel

- RULE: close(ch) is done by the writing goroutine when nothing is left to write; writing to or re-closing a

closed channel panics.

- Reading from a closed channel always succeeds (returns buffered values, then zero value).
- Closing is required only if a reader waits for close (e.g., for-range).
- Use comma ok (v, ok := <-ch) to distinguish a written zero value from a closed-channel zero value.
- GOTCHA: Writing to a closed channel or closing twice (panics).

Understanding How Channels Behave

- RULE: Table 12-1: nil channel read/write hangs forever; closed unbuffered read returns zero value; closed write panics.
- Avoid panics; make the writer responsible for closing.
- Multiple writers to one channel complicate closing (use sync.WaitGroup).
- GOTCHA: none in text

select

- RULE: select chooses at random from ready channel cases; each case is its own block; default runs if no case is ready.
- Random choice prevents starvation and inconsistent lock-order deadlocks.
- A select with default gives a nonblocking read/write; default in a for-select loop spins the CPU (almost always wrong).
- for-select loops are common; include a way to exit.
- GOTCHA: Placing default inside a for-select loop (spins CPU).

Concurrency Practices and Patterns

- RULE: Keep concurrency out of your API; don't export channels or mutexes.
- Channels as params/fields are fine if unexported.
- GOTCHA: none in text

Keep Your APIs Concurrency-Free

- RULE: Hide concurrency implementation details so callers don't manage buffering, closing, or nil channels.
- Exception: concurrency helper library APIs use channels.
- GOTCHA: none in text

Goroutines, for Loops, and Varying Variables

- RULE: Before Go 1.22, closures in for loops captured the same index/value variable (all saw the last value).
- Go 1.22+ creates new index/value variables per iteration (control via go directive).
- Fix without Go 1.22: shadow (v := v) or pass the value as a parameter to the goroutine.
- Always pass a copy of any changing variable into a closure.
- GOTCHA: Goroutines seeing the last loop value (pre-1.22).

Always Clean Up Your Goroutines

- RULE: Every goroutine must eventually exit; leaked goroutines keep stack/heap memory from being GC'd.
- Exiting a loop early can leave a producer goroutine blocked forever.
- GOTCHA: Leaving a goroutine blocked on a channel write after the reader stops.

Use the Context to Terminate Goroutines

- RULE: Pass context.Context to goroutines; check <-ctx.Done() in a for-select loop to exit.
- Create cancel with context.WithCancel; defer cancel() so Done closes when the caller exits.
- GOTCHA: none in text

Know When to Use Buffered and Unbuffered Channels

- RULE: Buffered channels are useful when you know how many goroutines launched, want to limit goroutines, or want to limit queued work.
- Buffered channels never have unlimited buffers; handle the full-buffer case.
- GOTCHA: none in text

Implement Backpressure

- RULE: Use a buffered channel of tokens and select/default to limit simultaneous requests (return "no more capacity" when full).
- GOTCHA: none in text

Turn Off a case in a select

- RULE: To stop reading a closed channel in a select, set its variable to nil (a read from nil never returns).
- Track closed-channel count with comma ok.
- GOTCHA: none in text

Time Out Code

- RULE: Run work in a goroutine writing to a size-1 buffered channel; select between the result and <-ctx.Done() from context.WithTimeout.
- A timed-out goroutine keeps running unless you cancel via context.
- GOTCHA: none in text

Use WaitGroups

- RULE: sync.WaitGroup (Add, Done, Wait) waits for multiple goroutines; zero value is useful; capture by closure so all use one instance.
- Call Done via defer so it runs even on panic.
- Use WaitGroups only when you have cleanup (like closing a channel) after workers exit.
- sync.WaitGroup and sync.Once must never be copied; pass by pointer.
- GOTCHA: Copying a WaitGroup (lock won't be shared).

Run Code Exactly Once

- RULE: sync.Once.Do runs a closure once; sync.OnceValue/OnceValues/OnceFunc (Go 1.21+) return a cached function (zero/one/two return values).
- Don't declare sync.Once inside a function (new instance each call loses memory).
- GOTCHA: Declaring sync.Once inside a function.

Put Your Concurrent Tools Together

- RULE: Combine goroutines, channels, select, context timeouts, and WaitGroups into pipelines where independent parts run in any order and nothing hangs.
- GOTCHA: none in text

When to Use Mutexes Instead of Channels

- RULE: "Share memory by communicating; do not communicate by sharing memory."
- Use channels to coordinate goroutines or transform values; use mutexes to share access to a struct field.
- sync.Mutex (Lock/Unlock) and sync.RWMutex (RLock/RUnlock); always release with defer.
- Mutexes aren't reentrant (acquiring the same lock twice deadlocks); never copy mutexes.
- Never access a variable from multiple goroutines without a mutex (data races).
- GOTCHA: Acquiring a nonreentrant mutex twice (self-deadlock).

Atomics-You Probably Don't Need These

- RULE: sync/atomic gives CPU atomic operations (add, swap, load, store, CAS) for single-register values.
- For everyone except performance experts, use goroutines and mutexes.

- GOTCHA: none in text

Where to Learn More About Concurrency

- RULE: Katherine Cox-Buday's "Concurrency in Go" covers more patterns; recommended for deeper study.
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises build channel-communicating goroutines, for-select reads, and sync.OnceValue caching; solutions in Chapter 12 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You learned Go concurrency rules and patterns; next is the standard library.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
go f()	Launch a goroutine
make(chan T)	Create an unbuffered channel
make(chan T, n)	Create a buffered channel
<-ch / ch <- v	Read from / write to a channel
close(ch)	Close a channel (sender only)
v, ok := <-ch	Comma ok for closed channel
select { case <-ch: }	Pick a ready channel case
wg.Add(n); wg.Done(); wg.Wait()	Wait for goroutines
once.Do(f)	Run code once
ctx.Done()	Channel that closes on cancellation

SELF-TEST

1. Who is responsible for closing a channel?
2. What happens if you write to a closed channel?
3. What does select do when multiple cases are ready?
4. What does sync.WaitGroup's Wait method do?
5. Which keyword launches a goroutine?

ANSWERS

1. The goroutine that writes to the channel
2. It panics
3. Picks one at random
4. Pauses until the counter hits zero
5. go

CH13

Chapter 13: The Standard Library

SECTIONS COVERED

io and Friends

time

Monotonic Time

Timers and Timeouts

encoding/json

Using Struct Tags to Add Metadata

Unmarshaling and Marshaling

JSON, Readers, and Writers

Encoding and Decoding JSON Streams

Custom JSON Parsing

net/http

The Client

The Server

ResponseController

Structured Logging

Exercises

Wrapping Up

GOAL

Use Go's standard library for I/O, time, JSON, and HTTP clients/servers.

KEY IDEAS

1. io.Reader reads into a buffer you pass in; io.Writer writes from a buffer. `io.Copy(dst, src)`
2. Use struct tags to map JSON fields; unmarshal into, marshal out of structs. `json:"name"`
3. http.Client needs a Timeout; http.Server needs Read/Write/Idle timeouts. `&http.Client{Timeout: 30 * time.Second}`
4. slog is the structured logger; log/slog is new in Go 1.21. `slog.Info("msg", "id", id)`
5. time.Now carries monotonic time; use time.Duration constants like time.Hour.

```
time.Since(start)
```

FULL NOTES

io and Friends

- RULE: io.Reader reads into a buffer you provide (Read(p []byte) (n int, err error)); io.Writer writes from a buffer.
- Reuse one buffer across Read calls to allocate once; check io.EOF to know reading is done.
- Process buf[:n] (the bytes actually read), not the whole buffer.
- io.Copy copies from an io.Reader to an io.Writer; io.MultiReader, io.LimitReader, io.MultiWriter combine readers/writers.
- io.Closer (Close() error) is usually called via defer; don't defer Close in a loop (it runs at function exit).
- io.Seeker supports random access with whence constants io.SeekStart/Current/End.
- Combined interfaces: io.ReadCloser, io.ReadWriter, etc.; use them to state intent.
- io.NopCloser adds a no-op Close to an io.Reader to make an io.ReadCloser (embedded type pattern).
- os.ReadFile/os.WriteFile/io.ReadAll are fine for small data; use os.Create/Open/OpenFile for large data (return *os.File).
- GOTCHA: deferring Close inside a loop.

time

- RULE: time.Duration is an int64 of nanoseconds with constants (time.Nanosecond … time.Hour); time.Time is a moment with a time zone.
- Don't use == on time.Time (time zones); use Equal.
- Go's format reference time: January 2, 2006 at 3:04:05PM MST (01/02 03:04:05PM '06 -0700).
- time.Time methods: Day, Month, Year, Hour, Minute, Second, After, Before, Equal, Sub, Add, AddDate, Truncate, Round (all value receivers).
- GOTCHA: Using == to compare time.Time instances.

Monotonic Time

- RULE: Go tracks monotonic time (counts from boot) for timers and time.Now; wall clock can jump (DST, NTP).
- Sub uses the monotonic clock if both times have it set; otherwise uses wall time.
- GOTCHA: none in text

Timers and Timeouts

- RULE: time.After returns a channel that fires once; time.Tick fires repeatedly (don't use outside trivial programs-the ticker can't be stopped).
- Use time.NewTicker instead (has Stop); time.AfterFunc runs a function after a duration.
- GOTCHA: Using time.Tick (ticker can't be shut down/GC'd).

encoding/json

- RULE: Marshal converts Go to JSON; Unmarshal converts JSON to Go; reflection powers it.
- Use struct tags (backtick strings) to map JSON fields; json:"name" or json:"name,omitempty" or json:"-" to ignore.
- Struct tags aren't compiler-validated but go vet checks them; json can't read unexported fields.
- Default (no tag): field name matches case-insensitively on unmarshal; uppercase first letter on marshal.
- GOTCHA: omitempty's "empty" isn't the zero value (zero struct isn't empty; zero-length slice/map is).

Using Struct Tags to Add Metadata

- RULE: Covered under encoding/json; struct tags control JSON field names and options.

- GOTCHA: none in text

Unmarshaling and Marshaling

- RULE: json.Unmarshal([]byte(data), &o) populates a struct; json.Marshal(o) writes a struct to JSON bytes.
- Passing a pointer lets you reuse the same struct (memory control).
- GOTCHA: none in text

JSON, Readers, and Writers

- RULE: json.Decoder reads JSON from an io.Reader; json.Encoder writes JSON to an io.Writer (avoids loading all data).
- Use Decode in a loop; break on io.EOF.
- GOTCHA: none in text

Encoding and Decoding JSON Streams

- RULE: Read multiple JSON objects one at a time with a json.Decoder loop; break on io.EOF.
- Encode multiple values to any io.Writer (e.g., bytes.Buffer).
- GOTCHA: none in text

Custom JSON Parsing

- RULE: Implement json.Marshaler and json.Unmarshaler to override default JSON handling (e.g., custom time formats).
- Use a Dup type (same underlying type, no methods) inside MarshalJSON/UnmarshalJSON to avoid infinite recursion/stack overflow.
- To limit JSON-format coupling, define separate JSON-aware and data-processing structs and copy between them.
- Don't use map[string]any beyond exploration; replace with concrete types.
- encoding/gob and net/rpc exist but aren't recommended (use gRPC).
- GOTCHA: Calling json.Marshal on a type with MarshalJSON causes infinite recursion unless you use a Dup type.

net/http

- RULE: net/http has a production HTTP/2 client and server.
- GOTCHA: none in text

The Client

- RULE: Create your own http.Client with a Timeout (don't use DefaultClient-it has no timeout).
- Reuse one client for the whole program (safe across goroutines).
- Build requests with http.NewRequestWithContext(ctx, method, url, body); body is an io.Reader or nil.
- Call client.Do(req) to get an *http.Response; read res.Body (io.ReadCloser) and defer res.Body.Close().
- Avoid the package-level Get/Head/Post functions (they use the default client without timeout).
- GOTCHA: Using http.DefaultClient (no timeout).

The Server

- RULE: http.Server listens for requests; a handler implements http.Handler (ServeHTTP(http.ResponseWriter, *http.Request)).
- Set Addr and ReadTimeout/WriteTimeout/IdleTimeout (defaults don't time out-handle malicious clients).
- ResponseWriter methods order: Header(), WriteHeader(statusCode), Write(body); skip WriteHeader for 200.
- Use http.NewServeMux as the router (meets http.Handler); register with Handle(path, handler) or HandleFunc(path, func).
- Go 1.22 supports verbs and wildcards: "GET /hello/{name}"; read with r.PathValue("name").
- Don't use package-level http.Handle/ListenAndServe/DefaultServeMux outside trivial tests (shared state,

no timeout config).

- ServeMux can nest other ServeMuxes with http.StripPrefix.
- GOTCHA: Using package-level DefaultServeMux functions.

ResponseController

- RULE: http.ResponseWriter is an interface that can't grow; http.ResponseController wraps it to add optional methods (Flush, Hijacker, SetReadDeadline, SetWriteDeadline).
- Check option support via http.NewResponseController(rw) and errors.Is(err, http.ErrNotSupported).
- This lets an interface evolve without breaking compatibility.
- GOTCHA: none in text

Structured Logging

- RULE: log/slog (Go 1.21) is the structured logger; slog.Info/Warn/Error/Debug with key-value pairs.
- slog.Debug is suppressed by default; control level via slog.HandlerOptions.
- Use slog.NewJSONHandler(os.Stderr, options) for JSON output to an io.Writer.
- For performance, use LogAttrs with slog.Attr (slog.String, slog.Time, slog.Any).
- log/slog API: context support, grouping, custom slog.Handler implementations; slog.NewLogLogger bridges to log.
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises: a time web server, JSON logging middleware, JSON vs text time via Accept header; solutions in Chapter 13 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You saw standard library packages embodying idiomatic Go; next is the context package.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
io.Copy(dst, src)	Copy reader to writer
json.Unmarshal(b, &o)	JSON to struct
json.Marshal(o)	Struct to JSON bytes
json.NewDecoder(r).Decode(&o)	Decode JSON from a reader
&http.Client{Timeout: ...}	HTTP client with timeout
http.NewServeMux()	HTTP request router
slog.Info("msg", "k", v)	Structured log with fields
time.Hour	Duration constant
time.NewTicker(d)	Repeating ticker you can stop

SELF-TEST

1. What indicates the end of data when reading from an io.Reader?
2. Which method should you not use to compare two time.Time values?
3. Which verb in a struct tag omits an empty field from JSON output?
4. What should you set on http.Client to avoid requests hanging forever?

5. Which package is Go's structured logger (added in Go 1.21)?

ANSWERS

1. io.EOF
2. ==
3. omitempty
4. Timeout
5. log/slog

CH14

Chapter 14: The Context

SECTIONS COVERED

What Is the Context?

Values

Cancellation

Contexts with Deadlines

Context Cancellation in Your Own Code

Exercises

Wrapping Up

GOAL

Pass request metadata, cancel work, and set deadlines through Go programs using context.Context.

KEY IDEAS

1. context.Context is an interface; pass it as the first function parameter named ctx.
2. context.WithValue adds key-value metadata; use unexported comparable keys to avoid collisions.

```
context.WithValue(ctx, key, val)
```

3. context.WithCancel returns a context and CancelFunc; call cancel via defer.

```
defer cancel()
```

4. context.WithTimeout/WithDeadline auto-cancel after a duration or time.Time.

```
context.WithTimeout(ctx, 2*time.Second)
```

5. Detect cancellation with <-ctx.Done() in select or context.Cause(ctx).

```
case <-ctx.Done():
```

FULL NOTES

What Is the Context?

- RULE: A context is an instance meeting the context.Context interface; pass it explicitly as the first function parameter (named ctx).
- Create an initial context with context.Background (entry point) or context.TODO (temporary placeholder; not for production).
- In HTTP servers, get context from req.Context() and create a new request with req.WithContext(ctx).
- For outgoing HTTP calls, use http.NewRequestWithContext(ctx, ...).
- GOTCHA: none in text

Values

- RULE: Prefer explicit parameters; use context values only when you can't pass data explicitly (e.g., HTTP middleware).
- context.WithValue(ctx, key, val) returns a child context wrapping the parent; contexts are immutable.
- Context never passes information out of deeper layers to higher layers.
- ctx.Value(key) does a linear search of the context chain; don't store dozens of values.
- Context keys must be comparable; don't use plain strings (collisions across packages).
- Use an unexported key type based on int with iota (multiple keys) or an empty struct (single key).
- Setter functions start with ContextWith; getter functions end with FromContext; return (value, bool).
- Copy context values into explicit parameters for business logic; keep system-maintenance info (tracking GUID) in the context.
- GOTCHA: Using a string as a context key (collisions).

Cancellation

- RULE: context.WithCancel returns a child context and a CancelFunc; cancel tells listeners to stop.
- Always call the cancel function when done (error or not); call via defer right after creation.
- Calling cancel multiple times is safe (no-ops after the first).
- ctx.Done() returns a chan struct{} closed on cancellation; reads from nil channel never return.
- A closed channel always returns its zero value immediately.
- Use context.WithCancelCause to attach an error to cancellation; context.Cause(ctx) retrieves it (Go 1.20+).
- ctx.Err() returns context.Canceled (explicit) or context.DeadlineExceeded (timeout).
- The standard library HTTP client respects cancellation.
- GOTCHA: Forgetting to call cancel (leaks memory and goroutines).

Contexts with Deadlines

- RULE: context.WithTimeout(ctx, duration) and context.WithDeadline(ctx, time.Time) auto-cancel after a limit.
- Child timeouts are bounded by the parent (a 3s child under a 2s parent cancels at 2s).
- ctx.Deadline() returns (time.Time, ok) like comma ok.
- Use context.WithTimeoutCause/WithDeadlineCause to return a custom sentinel error on timeout.
- Always defer the cancel functions from WithTimeout/WithDeadline too.
- GOTCHA: none in text

Context Cancellation in Your Own Code

- RULE: Pass context to other HTTP/database calls (libraries handle it).
- Handle cancellation in your code when reading/writing channels with select (include <-ctx.Done() case).
- For long-running loops, check context.Cause(ctx) periodically and return the error.
- GOTCHA: none in text

Exercises

- RULE: Chapter exercises: timeout middleware, summing until 1234 or 2s, log level in context; solutions in Chapter 14 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You can set timeouts, cancel explicitly, pass values, and know when to do each; next is testing.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
context.Background()	Empty initial context
context.WithValue(ctx, k, v)	Add metadata (child context)
ctx.Value(k)	Read a value from context
context.WithCancel(ctx)	Cancellable context + CancelFunc
defer cancel()	Ensure cancel is called
context.WithTimeout(ctx, d)	Auto-cancel after duration
<-ctx.Done()	Channel closed on cancellation
context.Cause(ctx)	Error that caused cancellation

SELF-TEST

1. Where should the context parameter go in a function signature?
2. Which function creates an initial empty context?
3. Why should context keys not be plain strings?
4. Which method returns a channel closed when the context is cancelled?
5. What does context.CancelFunc do?

ANSWERS

1. First parameter, named ctx
2. context.Background
3. They can collide across packages
4. ctx.Done()
5. Cancels the context

CH15

Chapter 15: Writing Tests

SECTIONS COVERED

Understanding the Basics of Testing
Reporting Test Failures
Setting Up and Tearing Down
Testing with Environment Variables
Storing Sample Test Data
Caching Test Results
Testing Your Public API
Using go-cmp to Compare Test Results
Running Table Tests
Running Tests Concurrently
Checking Your Code Coverage
Fuzzing
Using Benchmarks
Profiling Your Go Code
Using Stubs in Go
Mocks and Stubs
Using httptest
Using Integration Tests and Build Tags
Using the -short Flag
Finding Concurrency Problems with the Data Race Detector
Exercises
Wrapping Up

GOAL

Write Go tests, use table tests, coverage, benchmarks, fuzzing, stubs, httptest, and the race detector.

KEY IDEAS

1. Test files end in _test.go; test functions start with Test and take *testing.T named t.

```
func TestAdd(t *testing.T)
```

2. Report failures with t.Error/Errorf (continue) or t.Fatal/Fatalf (stop this test).

```
t.Error("msg")
```

3. Table tests use t.Run(name, func(t *testing.T)) for named subtests.

```
t.Run("addition", func(t *testing.T) { })
```

4. Benchmarks start with Benchmark, take *testing.B, and loop 0 to b.N.

```
func BenchmarkX(b *testing.B)
```

5. The data race detector runs with go test -race; it's ~10x slower.

```
go test -race
```

FULL NOTES

Understanding the Basics of Testing

- RULE: Tests live in _test.go files in the same package as production code; test funcs start with Test and take *testing.T (named t), return nothing.
- Run tests with go test; ./... runs current dir and all subdirs; -v gives verbose output.
- Use t.Error (like fmt.Print) to report failures; the test keeps running.
- GOTCHA: none in text

Reporting Test Failures

- RULE: t.Error/Errorf mark failed but continue; t.Fatal/Fatalf stop the current test function immediately (other tests still run).
- Use Fatal when further checks would always fail or panic; use Error for independent checks to report many problems at once.
- GOTCHA: none in text

Setting Up and Tearing Down

- RULE: TestMain(m *testing.M) runs once per package (not per test); set up, call m.Run(), os.Exit(exit code); only one TestMain per package.
- t.Cleanup(func()) registers per-test cleanup run LIFO when the test completes; useful with helper functions.
- t.TempDir() creates a temp directory per test and registers cleanup to delete it.
- GOTCHA: TestMain invoked once per package, not per test.

Testing with Environment Variables

- RULE: t.Setenv("VAR", "val") registers an env var for the test and reverts it via Cleanup when the test exits.
- Copy env vars into config structs in main so code is unaware of them and easier to test.
- GOTCHA: none in text

Storing Sample Test Data

- RULE: Put sample data in a testdata subdirectory; go test sets the working directory to the package, so use relative paths.
- GOTCHA: none in text

Caching Test Results

- RULE: go test caches passing test results across packages unless files changed; force rerun with -count=1.
- GOTCHA: none in text

Testing Your Public API

- RULE: Tests in package packagename_test (external test package) import and test only exported identifiers (black box).
- You can mix package and package_test test files in the same directory.
- GOTCHA: none in text

Using go-cmp to Compare Test Results

- RULE: cmp.Diff(expected, result) returns a string describing mismatches (empty if equal); use cmp.Comparer for custom field comparison (symmetric, deterministic, pure).
- GOTCHA: none in text

Running Table Tests

- RULE: Table tests use a slice of anonymous structs (name, inputs, expected) and t.Run for named subtests.
- Comparing error messages is fragile; use errors.Is/errors.As for custom/sentinel errors.

- GOTCHA: none in text

Running Tests Concurrently

- RULE: t.Parallel() as the first line runs a test concurrently with other parallel tests (speeds up suites); don't use with shared mutable state.
- Parallel tests panic if they also use t.Setenv.
- Parallel table tests capture loop variables; pre-Go 1.22 all see the last value (go vet flags this). Shadow the variable (d := d) or require Go 1.22+.
- GOTCHA: Capturing a loop variable in parallel table tests (pre-1.22).

Checking Your Code Coverage

- RULE: go test -cover shows coverage summary; -coverprofile=c.out saves it; go tool cover -html=c.out shows uncovered lines.
- 100% coverage is necessary but not sufficient (copy-paste bugs can hide).
- GOTCHA: none in text

Fuzzing

- RULE: Fuzz tests start with Fuzz, take *testing.F; add seed corpus with f.Add; call f.Fuzz(func(t *testing.T, in T)) with generated data.
- Corpus entry types limited to integers, floats, bool, string, []byte; invalid types to Add are a runtime error.
- Only one fuzz test can run at a time; failing inputs are saved to testdata/fuzz/TESTNAME as new seed corpus entries.
- Fuzzing can use lots of memory/disk; fuzz tests found allocation and blank-line bugs in ParseData.
- GOTCHA: none in text

Using Benchmarks

- RULE: Benchmark functions start with Benchmark, take *testing.B, loop 0 to b.N; run with go test -bench=. and -benchmem for memory info.
- Output columns: name-GOMAXPROCS, runs, ns/op, B/op, allocs/op.
- Use b.Run for sub-benchmarks; write results to a blackhole variable to prevent compiler optimization.
- All tests run before benchmarks; only benchmark when tests pass.
- GOTCHA: none in text

Profiling Your Go Code

- RULE: Go includes profiling tools (CPU/memory) and pprof visualization; beyond the book's scope.
- GOTCHA: none in text

Using Stubs in Go

- RULE: Abstract dependencies with interfaces/function types so tests can inject stubs (canned return values).
- Embed an interface in a stub struct to auto-define all methods; implement only what's needed for the test (calling an unimplemented method panics).
- For multiple inputs/outputs per test, use a proxy stub struct with function fields per method.
- A stub returns canned values; a mock validates call order/inputs.
- Popular mock libraries: gomock (Google) and testify (Stretchr).
- GOTCHA: Calling an unimplemented stub method (panics).

Mocks and Stubs

- RULE: Covered under Using Stubs; stub vs mock distinction.
- GOTCHA: none in text

Using httptest

- RULE: net/http/httptest.NewServer starts an HTTP server on a random port; use server.URL and server.Client(); defer server.Close().
- Good for testing HTTP clients without standing up real services.
- GOTCHA: none in text

Using Integration Tests and Build Tags

- RULE: Integration tests connect to real services; use custom build tags (//go:build integration) and run with go test -tags integration -v ./...; or env vars + t.Skip.
- GOTCHA: none in text

Using the -short Flag

- RULE: testing.Short() plus t.Skip marks slow tests; run with go test -short to skip them.
- The author finds -short unintuitive; build tags express dependencies better.
- GOTCHA: none in text

Finding Concurrency Problems with the Data Race Detector

- RULE: go test -race detects data races (unsynchronized shared variable access); -race binaries run ~10x slower.
- Fix races with locks, not sleeps.
- GOTCHA: Using sleeps to fix race conditions (still wrong).

Exercises

- RULE: Chapter exercises: unit tests to ~100% coverage, race detector fix, fuzz test against parser; solutions in Chapter 15 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You learned testing, coverage, benchmarking, fuzzing, race detection; next is reflect/unsafe/cgo.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
go test	Run tests in current package
go test ./...	Run tests in all subpackages
go test -v	Verbose output
t.Error(msg)	Report failure, keep running
t.Fatal(msg)	Stop current test function
t.Run(name, func)	Named subtest (table tests)
t.Parallel()	Run test concurrently
go test -cover -coverprofile=c.out	Coverage report
go tool cover -html=c.out	HTML coverage view
go test -fuzz=FuzzX	Fuzz test
go test -bench=. -benchmem	Run benchmarks with memory
go test -race	Data race detector

SELF-TEST

1. What suffix must a Go test file name end with?
2. Which method stops the current test function after a failure?
3. Which method creates a temp directory cleaned up after a test?
4. Which flag enables the data race detector?
5. Which function adds seed data to a fuzz test?

ANSWERS

1. _test.go
2. t.Fatal
3. t.TempDir
4. -race
5. f.Add

CH16

Chapter 16: Here Be Dragons: Reflect, Unsafe, and Cgo

SECTIONS COVERED

Reflection Lets You Work with Types at Runtime

Types, Kinds, and Values

Values

Make New Values

Use Reflection to Check If an Interface's Value Is nil

Use Reflection to Write a Data Marshaler

Build Functions with Reflection to Automate Repetitive Tasks

You Can Build Structs with Reflection, but Don't

Reflection Can't Make Methods

Use Reflection Only if It's Worthwhile

unsafe Is Unsafe

Using Sizeof and Offsetof

Using unsafe to Convert External Binary Data

Accessing Unexported Fields

Using unsafe Tools

Cgo Is for Integration, Not Performance

Exercises

Wrapping Up

GOAL

Use reflect, unsafe, and cgo as escape hatches only when normal Go can't solve the problem.

KEY IDEAS

1. Reflection inspects types/kinds/values at runtime but is slow and can panic; use at program boundaries. `reflect.TypeOf(v)`
2. unsafe.Sizeof/unsafe.Offsetof reveal memory layout and padding; reorder struct fields to save memory. `unsafe.Sizeof(Data{})`
3. cgo calls C from Go via import "C" and the C pseudopackage; it's slow (~29x) and only for integration. `C.add(3, 2)`

FULL NOTES

Reflection Lets You Work with Types at Runtime

- RULE: Reflection (reflect package) examines types/kinds/values at runtime and can read, modify, and create variables, functions, and structs.
- It's slower than normal code, more fragile and verbose; reflect methods panic on wrong kinds.
- Standard library uses reflection: database/sql, text/html/template, fmt, errors.Is/As, sort.Slice, encoding/json.
- leave comments explaining reflection code for future reviewers.
- GOTCHA: Calling a reflect method that doesn't match the kind (panics).

Types, Kinds, and Values

- RULE: reflect.TypeOf(v) returns reflect.Type (Name, Kind, NumIn, Elem, NumField, Field(i), Tag); reflect.Kind says what the type is made of (struct, slice, pointer, int, etc.).
- If you define struct Foo, kind is reflect.Struct and type name is "Foo".
- Elem on a reflect.Type of a pointer/slice/map/channel/array returns the contained type.
- GOTCHA: Name returns "" for unnamed types like slices and pointers.

Values

- RULE: reflect.ValueOf(v) returns reflect.Value; Interface() reads as any; type-assert back to usable type.
- Primitive read methods: Bool, Complex, Int, Uint, Float, String, Bytes (panic on mismatch); CanInt/CanFloat/etc. check first.
- To set a value: pass &v to reflect.ValueOf, use Elem(), then SetInt/SetString/SetBool/SetFloat/SetUint or Set.
- CanSet tells if setting will panic; setting without a pointer panics.
- GOTCHA: Setting a reflect.Value without passing a pointer (panics).

Make New Values

- RULE: reflect.New(typ) returns a pointer to a zero value of typ; reflect.MakeSlice/MakeMap/MakeChan create compound values.
- Create a reflect.Type without a value: reflect.TypeOf((*string)(nil)).Elem() or reflect.TypeOf([]string(nil)).
- reflect.Append(slice, value) appends; Interface() converts back to a Go value.
- GOTCHA: none in text

Use Reflection to Check If an Interface's Value Is nil

- RULE: IsValid() true unless reflect.Value holds a nil interface; IsNil() works only on kinds that can be nil (pointer, slice, map, func, interface) and panics otherwise.

- Strive to write code that works with nil interface values; reserve nil detection for last resorts.
- GOTCHA: Calling IsNil on a kind whose zero value isn't nil (panics).

Use Reflection to Write a Data Marshaler

- RULE: reflection reads struct tags (field.Tag.Lookup("csv")) and fields to marshal/unmarshal structs to [][]string.
- Marshal takes any (reading only); Unmarshal takes a pointer to a slice of structs (modifying).
- Switch on field.Kind() to convert types (Int->FormatInt, String, Bool->FormatBool).
- GOTCHA: none in text

Build Functions with Reflection to Automate Repetitive Tasks

- RULE: reflect.MakeFunc(ft, closure) generates a function wrapping another; return via Interface().
- Generated functions hide data flow; reflection invocation is slow; only worthwhile if the wrapped operation is already slow (e.g., network).
- GOTCHA: none in text

You Can Build Structs with Reflection, but Don't

- RULE: reflect.StructOf builds a struct type assignable only to any; fields read/written via reflection; mostly academic interest.
- GOTCHA: none in text

Reflection Can't Make Methods

- RULE: Reflection can create functions and struct types but cannot add methods to a type; can't create a type that implements an interface via reflection.
- GOTCHA: none in text

Use Reflection Only if It's Worthwhile

- RULE: Reflection is essential at data boundaries (CSV marshaling, memoizing); benchmark showed reflection ~50-75x slower than custom/generic with thousands of allocations.
- The compiler can't stop wrong types passed to reflection-based functions (runtime crash).
- Use generics instead when possible.
- GOTCHA: none in text

unsafe Is Unsafe

- RULE: unsafe.Pointer converts any pointer to/from any other pointer and to/from uintptr; enables pointer arithmetic and byte manipulation.
- Two patterns: type conversion via unsafe.Pointer bridge; reading/modifying bytes of a variable.
- Use requires knowing size/location; unsafe.Sizeof and unsafe.Offsetof provide this.
- 24% of studied Go projects use unsafe; mostly OS/C integration and efficiency.
- GOTCHA: none in text

Using Sizeof and Offsetof

- RULE: unsafe.Sizeof returns bytes of a type (slice=24 bytes, string=16, map=8 on 64-bit); unsafe.Offsetof returns field position within a struct.
- Struct size includes padding for alignment; reordering fields (group bools together) reduces padding and memory.
- Sizeof/Offsetof results are compile-time constants (usable in const).
- GOTCHA: none in text

Using unsafe to Convert External Binary Data

- RULE: Map network bytes into a struct whose in-memory layout matches the wire protocol using unsafe.Pointer; handle endianness (bits.ReverseBytes32 on little-endian).

- Use arrays (value types) not slices for mapping; unsafe.Slice/unsafe.SliceData convert between slices and structs.
- Benchmark: unsafe array conversion ~2-2.5x faster; slice conversion is slowest and allocates.
- For most programs, stick with safe code.
- GOTCHA: none in text

Accessing Unexported Fields

- RULE: Combine reflect (FieldByName for the offset) and unsafe (unsafe.Pointer + unsafe.Add) to read/modify unexported struct fields; use only as a last resort.
- GOTCHA: none in text

Using unsafe Tools

- RULE: Run with -gcflags=-d=checkptr to add runtime checks for unsafe.Pointer misuse (like race checker, not exhaustive, slows program).
- Avoid unsafe unless you know what you're doing and need the performance.
- GOTCHA: none in text

Cgo Is for Integration, Not Performance

- RULE: cgo is Go's FFI to C; import "C" with a preceding C comment block makes C functions/types/variables available as C.name, C.int, C.CString.
- Expose Go functions to C with //export; C code goes in .c files including "_cgo_export.h".
- Don't pass Go values containing pointers (strings, slices, functions, structs with them) directly to C; use cgo.Handle to wrap instances (cgo.NewHandle, h.Value(), h.Delete()).
- Can't call variadic C functions, C function pointers, or convert C unions cleanly.
- cgo calls are ~29x slower than C-to-C; only use cgo when you must use a C library with no Go replacement; prefer third-party wrappers.
- GOTCHA: Passing a pointer-containing Go value into C (crash/incorrect behavior when GC'd).

Exercises

- RULE: Chapter exercises: reflection string-length validator, unsafe Sizeof/Offsetof reordering, cgo mini_calc; solutions in Chapter 16 repository.
- GOTCHA: none in text

Wrapping Up

- RULE: You've completed the journey through Go; idiomatic Go is boring, straightforward, maintainable.
- GOTCHA: none in text

CHEAT SHEET

Syntax	What it does
---	---
reflect.TypeOf(v)	Get reflect.Type of a value
reflect.ValueOf(v)	Get reflect.Value of a value
iv.Elem().SetInt(20)	Set a value via reflection (pass pointer)
unsafe.Sizeof(x)	Size in bytes of a type
unsafe.Offsetof(x.f)	Field offset in a struct
(*Data)(unsafe.Pointer(&b))	Unsafe pointer conversion
C.add(3, 2)	Call C function via cgo
cgo.NewHandle(p)	Wrap a Go value to pass to C

SELF-TEST

1. What does reflect.Kind tell you about a type?
2. Which unsafe function returns the size of a type in bytes?
3. Can reflection add methods to a type?
4. Should you pass a Go string directly into a C function via cgo?
5. When is reflection roughly 50-75x slower than generics?

ANSWERS

1. What the type is made of (struct, slice, pointer, int, etc.)
2. unsafe.Sizeof
3. No
4. No (strings contain pointers)
5. Filtering 1,000-element slices with reflection vs generics/custom

CH6 EXERCISE NOTES

ch6 - Pointers: Notes

Short, scannable notes for Exercises 6.1-6.3. Designed to be read in small chunks.

Exercise 6.1 - Person Struct: Value vs Pointer

What it does: Creates a `Person` struct (FirstName, LastName, Age) with two constructor functions:

- `MakePerson` -> returns a `Person` value (a copy)
- `MakePersonPointer` -> returns a `*Person` pointer (an address)

Key difference:

	Value `Person`	Pointer `*Person`
---	---	---
What's copied	The whole struct	Just the address (8 bytes)
Memory	New copy each time	One struct, many pointers
Changes to copy	Don't affect original	Affect the original

How to call:

```
p1 := MakePerson("Bertin", "Dreyer", 28)      // value
p2 := MakePersonPointer("Bertin", "Dreyer", 28) // pointer

println(p1.FirstName, p1.LastName, p1.Age)
println(p2.FirstName, p2.LastName, p2.Age)   // Go auto-dereferences p2
```

When to use which:

- Use a value when you want an independent copy.
- Use a pointer when you want to share one struct or avoid copying large data.

Exercise 6.2 - Slice Pointers: UpdateSlice vs GrowSlice

What it does: Two functions that take a `[]string` :

- `UpdateSlice(slice, s)` - changes the last element in place.
- `GrowSlice(slice, s)` - appends a new element.

The gotcha: Slices are headers, not the data itself.

A slice header has 3 parts:

1. Pointer to the underlying array
2. Length
3. Capacity

When you pass a slice to a function, the header is copied, but the pointer still points to the same underlying array.

UpdateSlice (works on caller's data)

```
func UpdateSlice(slice []string, s string) {
    slice[len(slice)-1] = s   // modifies the shared array
}
```

The caller sees the change because the pointer inside the copied header still points to the same array.

GrowSlice (does NOT persist to caller)

```
func GrowSlice(slice []string, s string) {
    slice = append(slice, s)   // may create a NEW array
}
```

If `append` needs more room, it allocates a new array and updates the local header. The caller's header still points to the old array, so the caller doesn't see the new element.

Fix: Reassign in `main` :

```
words = GrowSlice(words, "date")
```

Output you'll see:

```
Before UpdateSlice: [apple banana carrot]
After UpdateSlice:  [apple banana cherry]   <- changed!

Before GrowSlice:   [apple banana cherry]
After GrowSlice:    [apple banana cherry]   <- NOT changed in main!
```

Exercise 6.3 - GC Tuning with 10 Million Entries

What it does: Builds a `[]Person` with 10,000,000 entries, times it, and prints how many GC runs happened.

Why it matters: Every time `append` reallocates the slice, it copies the old data to a new array. The old array becomes garbage. More garbage = more GC runs = slower.

GOGC: Controls how often GC runs.

- GOGC = percent of heap growth allowed before triggering GC.

- Default = 100.
- Higher GOGC -> fewer GC runs -> usually faster, but uses more memory.
- Lower GOGC -> more GC runs -> usually slower, but keeps memory lower.

How to test:

```
# Default (GOGC=100)
go run .

# Less frequent GC
$env:GOGC="500"; go run .

# More frequent GC
$env:GOGC="10"; go run .
```

Results (this machine, no pre-allocation):

GOGC	Time	GC Runs
------	------	---------
100 (default)	1.63s	15
500	1.20s	4 <- fastest
10	1.81s	30 <- slowest

Takeaway: Higher GOGC = faster here because fewer GC pauses. But it uses more memory between collections.

Pro tip: Pre-allocate capacity to avoid reallocations entirely:

```
people := make([]Person, 0, 10_000_000)
```

This usually makes it faster than any GOGC setting because there's no copying at all.

Quick Reference

- Struct value = full copy. Struct pointer = just an address.
- Slice header is copied by value, but points to shared data.
- Append may reallocate -> caller won't see new elements unless you reassign.
- GOGC trades memory for GC frequency. Higher = fewer GCs, usually faster.

End of notes. Export this to PDF if you want a printable version.